

CÂU SỐ 1 PCCP + TOÀN BỘ LEVEL 1 JS

86 bài phân tích • 84 bài Level 1 + 2 bài mô phỏng PCCP • JavaScript

Mục tiêu: Đọc nhiều bài Level 1 theo cùng một khuôn để nhìn thấy bộ xương lặp lại: Contract → Bound → Brute force → Bottleneck → Pattern → State → Transition → Invariant.

86 bài phân tích: 84 bài Level 1 hỗ trợ JavaScript + 2 bài mô phỏng PCCP bổ sung. Mỗi bài có đề diễn giải tiếng Việt, input/output, ràng buộc, pattern và code; không gồm SQL.

#	Bài	Nhóm	Pattern chính
1	Băng bó	Đề PCCP công khai	Event-based simulation
2	Trình phát video	Đề PCCP công khai	Simulation + chuẩn hóa thời gian
3	Chữ cái cô độc	Mô phỏng chính thức #1	Duyệt chuỗi + Set
4	Robot thực hành	Mô phỏng chính thức #2	Tọa độ + hướng

Cách dùng tài liệu

1. Đọc phần đề và tự viết Contract trước.
2. Nhìn Bound để đoán độ phức tạp cho phép.
3. Tự nghĩ cách ngây thơ, rồi mới đọc Pattern.
4. Che code và nói Transition bằng tiếng Việt.
5. Chạy tay một ví dụ trước khi gõ code.

Bài 1 — Băng bó

Nguồn đề chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/250137>

Đề bài — diễn giải đầy đủ

Một nhân vật có kỹ năng hồi máu theo chu kỳ. Kỹ năng được mô tả bởi $\text{bandage} = [t, x, y]$: trong mỗi giây không bị tấn công, nhân vật hồi x máu; nếu hồi liên tục đủ t giây thì nhận thêm y máu. Sau khi hoàn thành một chu kỳ t giây, chuỗi liên tục bắt đầu lại. Máu không bao giờ vượt quá health ban đầu.

Ở đúng giây quái vật tấn công, nhân vật không được hồi máu. Đòn đánh làm chuỗi hồi liên tục bị ngắt và trở về 0. Sau khi trừ sát thương, nếu máu ≤ 0 thì kết quả là -1. Nếu sống qua mọi đòn, trả lượng máu ngay sau đòn cuối.

Input / Output

Tham số	Ý nghĩa
$\text{bandage} = [t, x, y]$	t : độ dài chu kỳ; x : hồi mỗi giây; y : thưởng khi đủ chu kỳ
health	Máu tối đa và máu ban đầu
$\text{attacks}[i] = [\text{time}, \text{damage}]$	Thời điểm tấn công và sát thương; đã tăng dần theo time
return	Máu còn lại hoặc -1

Ràng buộc

- $1 \leq t \leq 50$; $1 \leq x, y \leq 100$
- $1 \leq \text{health} \leq 1.000$
- $1 \leq \text{attacks.length} \leq 100$
- $1 \leq \text{attack time} \leq 1.000$; $1 \leq \text{damage} \leq 100$
- Các thời điểm tấn công khác nhau và đã sắp xếp tăng dần

Ví dụ

bandage	health	attacks	Kết quả
[5,1,5]	30	[[2,10],[9,15],[10,5],[11,5]]	5
[3,2,7]	20	[[1,15],[5,16],[8,6]]	-1
[1,1,1]	5	[[1,2],[3,2]]	3

Phân tích theo khung tư duy

Contract

Tìm máu còn lại sau toàn bộ attacks; chết tại bất kỳ đòn nào thì trả -1.

Bound

Thời điểm cuối tối đa 1.000 nên mô phỏng từng giây $O(\text{lastTime})$ cũng đủ nhanh. Duyệt theo sự kiện $O(\text{attacks.length})$ còn gọn hơn.

Brute force

Từ giây 1 tới giây tấn công cuối: kiểm tra giây này có attack hay không; nếu không thì hồi và tăng chuỗi liên tục. Cách này đúng, không phải lời giải tệ với bound này.

Bottleneck

Không nằm ở tốc độ mà ở đúng thứ tự: giây tấn công không được hồi; attack reset chuỗi; máu bị chặn ở maxHealth; chết phải dừng ngay.

Pattern

Simulation theo sự kiện. Giữa hai attack không có sự kiện khác, nên tính thẳng cả khoảng hồi thay vì đi từng giây.

State

currentHealth và previousAttackTime. maxHealth là hằng số. recoveryTime chỉ là dữ liệu tạm của vòng hiện tại.

Transition

$\text{recoveryTime} = \text{attackTime} - \text{previousAttackTime} - 1$; $\text{totalHeal} = \text{recoveryTime} * x + \text{floor}(\text{recoveryTime}/t) * y$; chặn bằng maxHealth; sau đó trừ damage.

Invariant

Sau khi xử lý attacks[i], currentHealth đúng bằng máu ngay sau đòn thứ i; previousAttackTime đúng bằng thời điểm đòn đó.

Complexity

$O(\text{attacks.length})$ thời gian, $O(1)$ bộ nhớ.

Bẫy và edge case

- Quên -1 trong khoảng giữa hai attack.
- Hồi ở đúng giây bị đánh.
- Không dùng Math.min với máu tối đa.
- Dùng phần dư chu kỳ từ khoảng trước dù attack đã reset.
- Kiểm tra chết bằng < 0 thay vì ≤ 0 .

Concept mang sang bài khác: Nếu dữ liệu chỉ thay đổi tại các mốc sự kiện, hãy thử nhảy thẳng từ sự kiện này sang sự kiện kế tiếp và tính gộp cả khoảng.

Các bước giải

1. Tách t, x, y; lưu maxHealth và currentHealth.
2. Với mỗi attack, tính số giây yên bình giữa attack trước và attack này.
3. Tính hồi thường và số chu kỳ thưởng hoàn chỉnh.
4. Cộng hồi nhưng không vượt maxHealth.
5. Trừ damage; nếu currentHealth ≤ 0 thì trả -1.
6. Cập nhật previousAttackTime; hết vòng lặp trả currentHealth.

Code JavaScript

```
function solution(bandage, health, attacks) {
    const [castTime, healPerSecond, bonusHeal] = bandage;
    const maxHealth = health;
    let currentHealth = health;
    let previousAttackTime = 0;

    for (const [attackTime, damage] of attacks) {
        const recoveryTime = attackTime - previousAttackTime - 1;
        const normalHeal = recoveryTime * healPerSecond;
        const bonusCount = Math.floor(recoveryTime / castTime);
        const totalHeal = normalHeal + bonusCount * bonusHeal;

        currentHealth = Math.min(maxHealth, currentHealth + totalHeal);
        currentHealth -= damage;

        if (currentHealth <= 0) return -1;
        previousAttackTime = attackTime;
    }

    return currentHealth;
}
```

Chạy tay điểm mấu chốt

Nếu attack trước ở giây 2 và attack hiện tại ở giây 9, các giây hồi là 3,4,5,6,7,8: tổng $9-2-1 = 6$ giây. Với $t=5$, nhân vật nhận 6 lần hồi thường và $\text{floor}(6/5)=1$ lần thưởng.

Bài 2 — Trình phát video

Nguồn đề chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/340213>

Đề bài — diễn giải đầy đủ

Một trình phát video nhận chuỗi lệnh prev và next. prev lùi 10 giây nhưng không nhỏ hơn 00:00; next tiến 10 giây nhưng không vượt video_len. Nếu vị trí hiện tại nằm trong đoạn opening, kể cả đúng hai đầu mút, trình phát tự nhảy tới op_end.

Trước lệnh đầu tiên cũng phải áp dụng luật skip opening. Sau mỗi lệnh, vị trí mới lại được kiểm tra opening. Cuối cùng trả vị trí dạng mm:ss.

Input / Output

Tham số	Ý nghĩa
video_len	Độ dài video, mm:ss
pos	Vị trí ban đầu
op_start, op_end	Hai đầu đoạn opening
commands	Mảng prev/next
return	Vị trí cuối dạng mm:ss

Ràng buộc

- Các chuỗi thời gian có đúng 5 ký tự và dạng mm:ss
- $0 \leq \text{mm}, \text{ss} \leq 59$
- $\text{op_start} < \text{op_end}$; pos và op_end nằm trong video
- $1 \leq \text{commands.length} \leq 100$

Ví dụ

video_len	pos	opening	commands	Kết quả
34:33	13:00	00:55-02:55	[next, prev]	13:00
10:55	00:05	00:15-06:55	[prev,next,next]	06:55
07:22	04:05	00:15-04:07	[next]	04:17

Phân tích theo khung tư duy

Contract

Thực hiện tuần tự commands và trả vị trí hợp lệ cuối cùng ở dạng mm:ss.

Bound

Tối đa 100 lệnh; một lượt duyệt $O(\text{commands.length})$ là quá đủ.

Brute force

Giữ phút và giây riêng, mỗi lệnh tự xử lý mượn/tràn giây rồi ghép chuỗi. Vẫn làm được nhưng nhiều nhánh và dễ sai.

Bottleneck

Representation: mm:ss là chuỗi hiển thị, nhưng phép toán cần một số. Chưa chuẩn hóa dữ liệu thì code trở nên rối.

Pattern

Simulation + chuẩn hóa mọi thời điểm thành tổng số giây.

State

currentSeconds, videoLength, openingStart, openingEnd không đổi.

Transition

skip opening nếu $\text{start} \leq \text{current} \leq \text{end}$; prev dùng $\max(0, \text{current} - 10)$; next dùng $\min(\text{videoLength}, \text{current} + 10)$; sau lệnh lại skip.

Invariant

Sau mỗi lệnh, currentSeconds nằm trong $[0, \text{videoLength}]$ và không nằm ở bên trong đoạn opening (nếu chạm opening thì đã bằng op_end).

Complexity

$O(\text{commands.length})$ thời gian, $O(1)$ bộ nhớ.

Bẫy và edge case

- Chỉ skip opening sau lệnh mà quên vị trí ban đầu.
- Không tính hai đầu mút opening là thuộc đoạn.
- Lùi âm hoặc tiến quá video.
- Xử lý phép toán trực tiếp trên chuỗi.
- Quên padStart khi đổi lại mm:ss.

Concept mang sang bài khác: Khi dữ liệu có định dạng để hiển thị, hãy tìm một representation chuẩn hóa để tính toán: thời gian → giây, tiền → đơn vị nhỏ nhất, tọa độ → cặp số.

Code JavaScript

```
function solution(video_len, pos, op_start, op_end, commands) {
  const toSeconds = (time) => {
    const [minutes, seconds] = time.split(':').map(Number);
    return minutes * 60 + seconds;
  };

  const toTime = (totalSeconds) => {
    const minutes = Math.floor(totalSeconds / 60);
    const seconds = totalSeconds % 60;
    return `${String(minutes).padStart(2, '0')}:${String(seconds).padStart(2, '0')}`;
  };

  const videoLength = toSeconds(video_len);
  const openingStart = toSeconds(op_start);
  const openingEnd = toSeconds(op_end);
  let currentSeconds = toSeconds(pos);

  const skipOpening = () => {
    if (openingStart <= currentSeconds && currentSeconds <= openingEnd) {
      currentSeconds = openingEnd;
    }
  };

  skipOpening();
  for (const command of commands) {
    if (command === 'prev') currentSeconds = Math.max(0, currentSeconds - 10);
    else currentSeconds = Math.min(videoLength, currentSeconds + 10);
    skipOpening();
  }
  return toTime(currentSeconds);
}
```

Chạy tay

pos=00:05, opening=00:10-00:20, command=next. Đổi sang giây: current=5. next → 15. Vì $10 \leq 15 \leq 20$, skip → 20. Kết quả 00:20.

Bài 3 — Chữ cái cô độc

Nguồn đề chính thức: <https://school.programmers.co.kr/learn/courses/15008/lessons/121683>

Đề bài — diễn giải đầy đủ

Một chữ cái bị xem là cô độc khi nó xuất hiện trong ít nhất hai nhóm liên tiếp tách rời nhau. Số lần xuất hiện đơn thuần không đủ: aaaa chỉ là một nhóm nên không cô độc; aa...aa là hai nhóm nên a cô độc.

Với input_string chỉ gồm chữ thường, trả các chữ cô độc theo thứ tự alphabet, ghép thành một chuỗi. Nếu không có chữ nào, trả N.

Ràng buộc và ví dụ

- $1 \leq \text{input_string.length} \leq 2.600$; chỉ gồm a-z.

input_string	Kết quả	Lý do ngắn
edaaaabbccd	de	d và e xuất hiện ở các nhóm tách rời
eeddee	e	e có hai nhóm
string	N	mỗi chữ chỉ một nhóm
zbzbz	bz	b và z đều lặp lại theo nhóm

Phân tích theo khung tư duy

Contract

Tìm các ký tự bắt đầu từ hai nhóm trở lên; sắp xếp và ghép, hoặc trả N.

Bound

$n \leq 2.600$. $O(n)$, $O(26n)$, thậm chí vài cách đơn giản khác đều đủ; $O(n)$ giúp học đúng pattern.

Brute force

Với từng chữ a-z, quét chuỗi và đếm số lần bắt đầu nhóm của chữ đó. $O(26n)$, vẫn chạy tốt vì 26 là hằng số.

Bottleneck

Đề hỏi số nhóm, không hỏi tần suất. Map đếm a xuất hiện 4 lần không phân biệt aaaa với aa...aa.

Pattern

Duyệt chuỗi, phát hiện ranh giới nhóm bằng `currentChar !== previousChar`, dùng Set ghi nhớ nhóm đã gặp.

State

seenGroups, lonely, previousChar.

Transition

Chỉ khi bắt đầu nhóm mới: nếu char đã có trong seenGroups thì thêm vào lonely; nếu chưa thì thêm vào seenGroups. Sau đó cập nhật previousChar.

Invariant

Sau vị trí i, seenGroups chứa mọi chữ đã bắt đầu ít nhất một nhóm; lonely chứa mọi chữ đã bắt đầu từ hai nhóm trở lên.

Complexity

$O(n + 26 \log 26) \approx O(n)$ thời gian; $O(26) \approx O(1)$ bộ nhớ.

Bẫy và edge case

- Dùng Map đếm tần suất rồi kết luận sai.

- Xử lý mọi ký tự thay vì chỉ đầu nhóm.
- Thêm trùng vào đáp án; Set giải quyết việc này.
- Quên sort alphabet.
- Không có đáp án nhưng trả chuỗi rỗng thay vì N.

Concept mang sang bài khác: Các từ khóa liên tiếp, đoạn, cụm, nhóm, bị ngắt thường báo hiệu cần so sánh phần tử hiện tại với phần tử trước.

Code JavaScript

```
function solution(input_string) {
  const seenGroups = new Set();
  const lonely = new Set();
  let previousChar = '';

  for (const currentChar of input_string) {
    const startsNewGroup = currentChar !== previousChar;
    if (startsNewGroup) {
      if (seenGroups.has(currentChar)) lonely.add(currentChar);
      else seenGroups.add(currentChar);
    }
    previousChar = currentChar;
  }

  if (lonely.size === 0) return 'N';
  return [...lonely].sort().join('');
}
```

Chạy tay: eeddee

Ký tự	Nhóm mới?	seenGroups	lonely
e	Có	{e}	{}
e	Không	{e}	{}
d	Có	{e,d}	{}
d	Không	{e,d}	{}
e	Có	{e,d}	{e}
e	Không	{e,d}	{e}

Bài 4 — Robot thực hành

Nguồn đề chính thức: <https://school.programmers.co.kr/learn/courses/15009/lessons/121687>

Đề bài — diễn giải đầy đủ

Robot bắt đầu ở tọa độ (0,0), quay mặt theo hướng +y (Bắc). Chuỗi command được thực hiện từ trái sang phải. R quay phải 90°, L quay trái 90°, G tiến một ô theo hướng đang nhìn, B lùi một ô ngược hướng đang nhìn. Sau tất cả lệnh, trả [x,y].

Ràng buộc và ví dụ

- $1 \leq \text{command.length} \leq 1.000.000$
- Chuỗi chỉ gồm R, L, G, B
- Thứ tự ký tự chính là thứ tự thực hiện

command	Kết quả
GRGLGRG	[2,2]

command	Kết quả
GRGRGRB	[2,0]

Phân tích theo khung tư duy

Contract

Mô phỏng toàn bộ command và trả tọa độ cuối [x,y].

Bound

n có thể tới 1.000.000. Phải xử lý mỗi ký tự $O(1)$, tổng $O(n)$; không được tạo thao tác lồng nhau theo n.

Brute force

Viết bốn nhánh cho mỗi hướng trong từng lệnh G/B, cộng thêm nhiều nhánh cho R/L. Đúng nhưng dài và dễ đảo dấu.

Bottleneck

Biểu diễn hướng. Nếu hướng được mã hóa tốt, cả quay lẫn di chuyển đều trở thành phép toán nhỏ, nhất quán.

Pattern

Simulation tọa độ + direction vectors. directions theo chiều kim đồng hồ: Bắc, Đông, Nam, Tây.

State

x, y, directionIndex. Mảng directions là hằng số.

Transition

R: $(dir+1)\%4$. L: $(dir+3)\%4$ để tránh số âm. G: cộng vector hướng. B: trừ vector hướng.

Invariant

Sau khi xử lý i ký tự, (x,y) và directionIndex đúng là vị trí và hướng của robot sau i lệnh đầu tiên.

Complexity

$O(command.length)$ thời gian, $O(1)$ bộ nhớ.

Bẫy và edge case

- Nhầm x/y: Bắc là [0,1], Đông là [1,0].
- Sắp directions sai chiều rồi quay R/L bị đảo.
- Dùng $(dir-1)\%4$ trong JS và nhận -1.
- Lùi B nhưng lại quay robot 180° ; B chỉ di chuyển ngược, hướng không đổi.
- Dùng split tạo mảng 1 triệu ký tự không cần thiết; for...of duyệt trực tiếp được.

Concept mang sang bài khác: Khi state có hướng hữu hạn, hãy mã hóa hướng bằng index và vector thay vì viết hàng loạt if/else cho từng hướng.

Bảng hướng

directionIndex	Hướng	dx	dy
0	Bắc (+y)	0	1
1	Đông (+x)	1	0
2	Nam (−y)	0	-1
3	Tây (−x)	-1	0

Code JavaScript

```
function solution(command) {
  const directions = [
    [0, 1], // Bắc
    [1, 0], // Đông
    [0, -1], // Nam
    [-1, 0], // Tây
  ];

  let x = 0;
  let y = 0;
  let directionIndex = 0;

  for (const currentCommand of command) {
    if (currentCommand === 'R') {
      directionIndex = (directionIndex + 1) % 4;
    } else if (currentCommand === 'L') {
      directionIndex = (directionIndex + 3) % 4;
    } else {
      const [dx, dy] = directions[directionIndex];
      if (currentCommand === 'G') {
        x += dx;
        y += dy;
      } else {
        // B
        x -= dx;
        y -= dy;
      }
    }
  }
  return [x, y];
}
```

Chạy tay: GRGLGRG

Lệnh	Vị trí	Hướng
Start	(0,0)	Bắc
G	(0,1)	Bắc
R	(0,1)	Đông
G	(1,1)	Đông
L	(1,1)	Bắc
G	(1,2)	Bắc
R	(1,2)	Đông
G	(2,2)	Đông

Tổng hợp pattern sau 4 bài

Bốn đề khác bối cảnh nhưng cùng bộ xương: duyệt dữ liệu đúng thứ tự, giữ state, đọc sự kiện/lệnh mới, cập nhật state, giữ invariant, rồi trả state cuối.

Bài	Input được duyệt	State tối thiểu	Dấu hiệu nhận diện
Băng bó	attacks	health, previousTime	Sự kiện theo mốc thời gian
Trình phát video	commands	currentSeconds	Lệnh tuần tự + thời gian
Chữ cái cô độc	characters	2 Set + previousChar	Nhóm liên tiếp bị tách
Robot	commands	x,y,direction	Tọa độ + quay/di chuyển

Checklist tự phân tích một bài Level 1

- Contract: Input là gì? Output chính xác là gì?
- Bound: n lớn nhất bao nhiêu? $O(n^2)$ có chịu được không?
- Brute force: cách tự nhiên nhất là gì? Có thật sự chậm không?
- Bottleneck: chậm hoặc dễ sai ở bước nào?
- Pattern: để đang mô tả lệnh, sự kiện, nhóm, đếm hay tìm kiếm?
- State: sau mỗi bước, cần nhớ tối thiểu những gì?
- Transition: phần tử mới làm state đổi ra sao?
- Invariant: sau i bước, mỗi biến đang có ý nghĩa chính xác gì?
- Edge cases: đầu/cuối, bằng biên, rỗng, chết/không tồn tại, giới hạn tối đa.

Kết luận: Câu 1 PCCP công khai chủ yếu kiểm tra khả năng đọc đề, biểu diễn dữ liệu và mô phỏng chính xác. Chưa cần Binary Search, DFS hay DP; nhưng phải kiểm soát state và biên rất chắc.

Nguồn chính thức

- Băng bó: <https://school.programmers.co.kr/learn/courses/30/lessons/250137>
- Trình phát video: <https://school.programmers.co.kr/learn/courses/30/lessons/340213>
- Chữ cái cô độc: <https://school.programmers.co.kr/learn/courses/15008/lessons/121683>
- Robot thực hành: <https://school.programmers.co.kr/learn/courses/15009/lessons/121687>

Ghi chú bản quyền: phần đề trong tài liệu là bản diễn giải tiếng Việt đầy đủ ý để học, không phải bản sao nguyên văn dài từ Programmers.

PHẦN II — TOÀN BỘ LEVEL 1 JAVASCRIPT TRÊN PROGRAMMERS

Roster chuẩn hoá • 84 bài • đối chiếu 03/08/2026

Phạm vi: toàn bộ bài Level 1 hiện xuất hiện khi lọc Level 1 + JavaScript trong mục Coding Test Practice của Programmers. Không trộn bài SQL. Hai bài PCCP “Băng bó” và “Trình phát video” đã được phân tích sâu ở Phần I; 82 thẻ dưới đây hoàn tất roster. Hai lesson mô phỏng “Chữ cái cô độc” và “Robot thực hành” vẫn được giữ vì hữu ích cho câu 1 PCCP dù không nằm trong roster công khai 84 bài.

Chuẩn đọc một thẻ

Contract → Pattern → State/Transition → Complexity → Bẫy → Code. Hãy che phần code, tự nói invariant, rồi mới đối chiếu.

Bản đồ pattern Level 1

- Scan / reduce / arithmetic: tổng, min/max, parity, chữ số, dot product, công thức.
- String / parsing: chuẩn hoá chuỗi, token, regex, mã hoá ký tự, cửa sổ con.
- Map / Set / counting: tần suất, khử trùng, last position, index lookup.
- Simulation / state machine: thời gian, lệnh, lịch, game, stack giả lập.
- Sort / greedy: sắp thứ tự để quyết định đơn điệu, tie-break nhiều khóa.
- Brute force nhỏ: cặp/bộ ba, mẫu tuần hoàn, thử hữu hạn.
- Matrix / coordinate: duyệt lưới, bounding box, zigzag, bitmask map.
- Number theory: divisor, prime sieve, gcd/lcm, base conversion.
- Stack / queue / top-k: xoá mẫu, mô phỏng khay, duy trì k phần tử tốt nhất.

Quy tắc coverage

Mỗi URL dùng dạng /learn/courses/30/lessons/{id}?language=javascript. Tên Hàn được giữ trong ngoặc để tìm đúng bài. Code ưu tiên rõ invariant và an toàn với JavaScript; không dùng thư viện ngoài.

Bài 5 — Đèn giao thông vàng (노란불 신호등)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/468371?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Trên một con đường có n đèn tín hiệu. Mỗi đèn bắt đầu ở màu xanh tại giây 1 rồi lặp mãi theo thứ tự xanh, vàng, đỏ; thời lượng ba màu của từng đèn có thể khác nhau. Sự cố mất điện xảy ra đúng lúc tất cả đèn đồng thời ở màu vàng. Hãy tìm thời điểm sớm nhất xảy ra trạng thái này; nếu trong toàn bộ chu kỳ chung không bao giờ xảy ra thì trả -1. Ví dụ với hai chu kỳ [2,1,2] và [5,1,1], lần đầu cùng vàng là giây 13.

Input / Output

Nhận signals, mỗi phần tử [G,Y,R] là số giây xanh, vàng, đỏ của một đèn. Trả một số nguyên là giây sớm nhất hoặc -1.

Ràng buộc và lưu ý

- $2 \leq n \leq 5$; $1 \leq G,Y,R \leq 18$; $3 \leq G+Y+R \leq 20$. Thời gian tính từ 1, nên offset trạng thái tại giây t là $(t-1)$ theo chu kỳ.

Contract

Tìm giây sớm nhất mà tất cả đèn cùng vàng, hoặc -1 nếu không tồn tại.

Pattern và dấu hiệu nhận diện

Chu kỳ + LCM + mô phỏng theo thời gian. Khi nhiều hệ lặp, toàn bộ trạng thái lặp lại sau LCM các chu kỳ.

State • Transition • Invariant

Chu kỳ đèn i là $G+Y+R$. Duyệt $t=1..LCM$; $offset=(t-1)\%cycle$. Đèn vàng khi $G \leq offset < G+Y$. Invariant: trước khi sang $t+1$, mọi trạng thái tại t đã được kiểm tra đúng.

Complexity

$O(n \cdot LCM)$, $O(1)$; với tối đa 5 chu kỳ không quá 20.

Bẫy / edge case

- Sai mốc vì thời gian bắt đầu từ giây 1; quên rằng khoảng vàng là nửa mở; dừng trước LCM.

Code JavaScript

```
function solution(signals) {
  const gcd = (a, b) => (b ? gcd(b, a % b) : a);
  const lcm = (a, b) => (a / gcd(a, b)) * b;
  const limit = signals.reduce((v, s) => lcm(v, s[0] + s[1] + s[2]), 1);
  for (let t = 1; t <= limit; t++)
    if (
      signals.every(([g, y, r]) => {
        const x = (t - 1) % (g + y + r);
        return g <= x && x < g + y;
      })
    )
      return t;
  return -1;
}
```

Bài 6 — Chống spoiler từ quan trọng (중요한 단어를 스포 방지)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/468370?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Một message gồm các từ chữ thường và chữ số, ngăn cách bởi một khoảng trắng. Một số đoạn ký tự không giao nhau được che spoiler và được mở lần lượt từ trái sang phải theo danh sách range. Chỉ cần một ký tự của từ nằm trong range thì từ đó là "từ spoiler"; nếu từ chạm nhiều range, nó chỉ lộ hoàn toàn khi range cuối cùng chạm nó được mở. Khi một từ vừa lộ hoàn toàn, từ đó quan trọng nếu không từng xuất hiện ở phần không che và chưa trùng bất kỳ từ spoiler đã lộ trước đó. Nếu nhiều từ cùng lộ, xét từ trái sang phải. Hãy đếm số từ quan trọng.

Input / Output

Nhận message và spoiler_ranges gồm các đoạn [start,end] inclusive đã tăng theo start. Trả số từ quan trọng.

Ràng buộc và lưu ý

- $1 \leq |message| \leq 20.000$; tối đa 1.000 range; range không chồng nhau. Một range có thể phủ nhiều từ và một từ có thể giao nhiều range; khoảng trắng cũng có thể nằm trong range.

Contract

Đếm từ spoiler khi vừa lộ hoàn toàn, chưa từng xuất hiện ở vùng thường và chưa trùng từ spoiler đã lộ trước.

Pattern và dấu hiệu nhận diện

Parsing span + sweep theo thời điểm reveal + Set. Một từ có thể cắt qua nhiều range nên thời điểm lộ hoàn toàn là range muộn nhất chạm nó.

State • Transition • Invariant

Đánh dấu mỗi index bằng id range; regex lấy word và span. maxRange=-1 nghĩa từ thường, đưa vào plain. Nhóm từ spoiler theo maxRange; duyệt nhóm theo range rồi trái→phải, kiểm tra plain/seen và luôn thêm từ vào seen.

Complexity

$O(|message| + \text{số ký tự trong các word})$, $O(|message|)$.

Bẫy / edge case

- Chỉ xét range đầu chạm word; chỉ lưu các từ quan trọng thay vì mọi spoiler word đã lộ; phá thứ tự trái→phải trong cùng lượt.

Code JavaScript

```
function solution(message, ranges) {
  const at = Array(message.length).fill(-1);
  ranges.forEach(([l, r], i) => {
    for (let p = l; p <= r; p++) at[p] = i;
  });
  const plain = new Set(),
    groups = Array.from({ length: ranges.length }, () => []);
  for (const m of message.matchAll(/[a-z0-9]+/g)) {
    let k = -1;
    for (let p = m.index; p < m.index + m[0].length; p++) k = Math.max(k, at[p]);
    k < 0 ? plain.add(m[0]) : groups[k].push(m[0]);
  }
  const seen = new Set();
  let ans = 0;
  for (const g of groups)
    for (const w of g) {
      if (!plain.has(w) && !seen.has(w)) ans++;
      seen.add(w);
    }
  return ans;
}
```

Chạy tay hai tình huống

Ví dụ 1: message = “here is muzi here is a secret message”, ranges = [[0,3],[23,28]]. “here” lộ ở range đầu nhưng đã xuất hiện trong vùng không che nên không quan trọng; “secret” lộ ở range sau, không có bản thường và chưa từng lộ, nên kết quả là 1.

Ví dụ 2: khi các range lần lượt làm lộ phone, 01012345678, may, i rồi phone, number, bốn từ đầu tiên đủ điều kiện. phone lần sau bị loại vì đã lộ; number bị loại vì có bản xuất hiện ở vùng không che. Kết quả là 4.

Test biên phải tự tạo

- Range chỉ che một ký tự giữa từ; một từ chạm hai range; hai từ giống nhau cùng lộ trong một lượt; từ spoiler có một bản giống hệt ở trước, giữa hoặc sau các range nhưng không bị che.

Bài 7 — Lấy hộp hàng (택배 상자 꺼내기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/389478?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Các hộp được đánh số $1..n$ và xếp thành tầng rộng w theo đường zíc-zắc: tầng đầu trái sang phải, tầng kế phải sang trái, rồi lặp lại. Muốn lấy hộp num phải lấy trước mọi hộp đang nằm phía trên nó trong cùng cột. Hãy đếm tổng số hộp phải lấy, bao gồm chính num . Ví dụ $n=22, w=6, num=8$ thì cần lấy 20, 17 rồi 8, kết quả 3.

Input / Output

Nhận ba số nguyên n, w, num . Trả số hộp cần lấy.

Ràng buộc và lưu ý

- $2 \leq n \leq 100$; $1 \leq w \leq 10$; $1 \leq num \leq n$. Tầng cuối có thể chưa đủ w hộp và hướng xếp phải đảo sau mỗi tầng.

Contract

Đếm hộp phải lấy, gồm hộp num và mọi hộp nằm phía trên cùng cột.

Pattern và dấu hiệu nhận diện

Grid zigzag + mô phỏng cột. Số hộp nhỏ nên ánh xạ số $\rightarrow (row, col)$ trực tiếp để chứng minh hơn công thức khó đọc.

State • Transition • Invariant

Hàm $col(x)$ dùng $row = \text{floor}((x-1)/w)$, $pos = (x-1) \% w$; row chẵn đi trái $\rightarrow$ phải, row lẻ đảo cột. Duyệt x từ num tới n và đếm cùng cột.

Complexity

$O(n), O(1)$; $n \leq 100$.

Bẫy / edge case

- Quên đảo chiều ở tầng lẻ; nhầm row 1-based; chỉ cộng các số cách nhau w dù zigzag đổi cột.

Code JavaScript

```
function solution(n, w, num) {
  const col = (x) => {
    const q = Math.floor((x - 1) / w),
      r = (x - 1) % w;
    return q % 2 ? r : w - 1 - r;
  };
  const target = col(num);
  let answer = 0;
  for (let x = num; x <= n; x++) if (col(x) === target) answer++;
  return answer;
}
```


Bài 8 — Chế độ làm việc linh hoạt (유연근무제)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/388351?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi nhân viên đăng ký giờ đến làm mong muốn và trong một tuần được coi là đúng giờ nếu vào ứng dụng không muộn hơn giờ đăng ký cộng 10 phút ở tất cả các ngày làm việc. Thứ Bảy và Chủ nhật không ảnh hưởng kết quả. Thời gian được ghi bằng HHMM như 958 nghĩa là 09:58; vì vậy phải cộng phút theo hệ 60. Biết ngày bắt đầu của chuỗi 7 log, hãy đếm nhân viên đạt điều kiện.

Input / Output

Nhận schedules (giờ đăng ký của từng người), timelogs[i][0..6] và startday (1=Thứ Hai,...,7=Chủ nhật). Trả số nhân viên nhận thưởng.

Ràng buộc và lưu ý

- $1 \leq$ số nhân viên ≤ 1.000 ; mỗi người có đúng 7 log. Bỏ qua log rơi vào cuối tuần; 09:58 + 10 phút là 10:08, không phải 09:68.

Contract

Đếm nhân viên không đi muộn vào bất kỳ ngày làm việc nào trong 7 ngày.

Pattern và dấu hiệu nhận diện

Chuyển hoá HHMM → phút + simulation lịch tuần. Dữ liệu hiển thị dạng 958 không thể cộng 10 trực tiếp.

State • Transition • Invariant

Đổi lịch mong muốn và log sang phút. Với offset d, weekday=(startday-1+d)%7; bỏ qua 5,6. Nhân viên đạt nếu mọi weekday có log ≤ schedule+10.

Complexity

$O(7n)=O(n)$, $O(1)$ ngoài input.

Bẫy / edge case

- Cộng 10 vào 958 thành 968; tính sai modulo Chủ nhật; kiểm tra cả cuối tuần.

Code JavaScript

```
function solution(schedules, timelogs, startday) {
    const mins = (t) => Math.floor(t / 100) * 60 + (t % 100);
    let answer = 0;
    for (let i = 0; i < schedules.length; i++) {
        let ok = true,
            limit = mins(schedules[i]) + 10;
        for (let d = 0; d < 7; d++) {
            const day = (startday - 1 + d) % 7;
            if (day < 5 && mins(timelogs[i][d]) > limit) ok = false;
        }
        if (ok) answer++;
    }
    return answer;
}
```

Bài 9 — Người nhận quà nhiều nhất (가장 많이 받은 선물)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/258712?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Có danh sách bạn bè và lịch sử tặng quà dạng “người_gửi người_nhận”. Tháng sau, với mỗi cặp A,B: nếu số lần A tặng B khác số lần B tặng A, người tặng nhiều hơn sẽ nhận một quà từ người kia. Nếu hai chiều bằng nhau, so chỉ số quà = tổng đã tặng - tổng đã nhận; người có chỉ số lớn hơn nhận quà. Nếu vẫn hòa thì không ai nhận. Hãy tính số quà tháng sau của từng người và trả giá trị lớn nhất.

Input / Output

Nhận friends gồm tên duy nhất và gifts gồm các chuỗi quan hệ. Trả số quà lớn nhất một người sẽ nhận.

Ràng buộc và lưu ý

- Xét mỗi cặp đúng một lần; lịch sử có thể lặp cùng cặp nhiều lần. Cần phân biệt ma trận tặng hai chiều và chỉ số quà toàn cục.

Contract

Tính số quà tháng sau lớn nhất mà một người nhận theo lịch sử cho/nhận và gift index.

Pattern và dấu hiệu nhận diện

Map tên→index + ma trận quan hệ + tie-break. Khi luật phụ thuộc từng cặp, xét mỗi cặp $i < j$ đúng một lần.

State • Transition • Invariant

Ma trận $a[i][j]$ đếm i cho j ; idx =cho-nhận. Với mỗi cặp, so hai chiều; hòa dùng idx ; hòa tiếp thì không ai nhận. next lưu số quà dự kiến.

Complexity

$O(gifts + n^2)$, $O(n^2)$.

Bẫy / edge case

- Xét cặp hai lần; gift index đảo dấu; hòa cả hai điều kiện vẫn cộng quà.

Code JavaScript

```
function solution(friends, gifts) {
    const n = friends.length,
          id = new Map(friends.map((x, i) => [x, i]));
    const a = Array.from({ length: n }, () => Array(n).fill(0)),
          idx = Array(n).fill(0),
          next = Array(n).fill(0);
    for (const s of gifts) {
        const [x, y] = s.split(' ').map((v) => id.get(v));
        a[x][y]++;
        idx[x]++;
        idx[y]--;
    }
    for (let i = 0; i < n; i++)
        for (let j = i + 1; j < n; j++) {
            if (a[i][j] > a[j][i] || (a[i][j] === a[j][i] && idx[i] > idx[j])) next[i]++;
            else if (a[j][i] > a[i][j] || (a[i][j] === a[j][i] && idx[j] > idx[i])) next[j]++;
        }
    return Math.max(...next);
}
```

Bài 10 — Cuộc đua chạy (달리기 경주)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/178871?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

players là mảng xếp hạng hiện tại từ hạng cao tới thấp. Mỗi tên trong callings cho biết người đó vừa vượt đúng người đứng ngay trước mình. Hãy xử lý toàn bộ lời gọi theo thứ tự và trả mảng xếp hạng cuối. Ví dụ [mumu,soe,poe,kai,mine] với hai lần gọi kai sẽ đưa kai tiến dần từng hạng, không được nhảy trực tiếp hai vị trí trong một transition.

Input / Output

Nhận players và callings. Trả mảng tên theo thứ hạng sau cùng.

Ràng buộc và lưu ý

- Tối đa 50.000 người và 1.000.000 lời gọi; người hạng nhất không bị gọi. Dùng tìm tuyến tính cho mỗi lời gọi sẽ quá chậm.

Contract

Xử lý mỗi lần một người vượt người ngay trước và trả mảng xếp hạng cuối.

Pattern và dấu hiệu nhận diện

Simulation + Map vị trí. Mảng giữ thứ tự, Map cho lookup $O(1)$.

State • Transition • Invariant

position[name] luôn khớp index trong players. Mỗi calling: lấy i, swap i/i-1, cập nhật đúng hai entry. Invariant này loại indexOf lặp.

Complexity

$O(\text{players} + \text{callings})$, $O(\text{players})$.

Bẫy / edge case

- Chỉ swap mảng mà quên Map; dùng indexOf cho tới 1.000.000 lời gọi.

Code JavaScript

```
function solution(players, callings) {
  const pos = new Map(players.map((x, i) => [x, i]));
  for (const x of callings) {
    const i = pos.get(x),
          y = players[i - 1];
    [players[i - 1], players[i]] = [x, y];
    pos.set(x, i - 1);
    pos.set(y, i);
  }
  return players;
}
```

Bài 11 — Điểm kỷ niệm (추억 점수)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/176963?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi người có một điểm nhớ thương. Một photo là danh sách tên xuất hiện trong ảnh; điểm của ảnh bằng tổng điểm của các tên có trong bảng, còn tên không biết tính 0. Hãy tính điểm cho từng ảnh theo đúng thứ tự đầu vào.

Input / Output

Nhận name, yearning song song và photo là mảng các mảng tên. Trả mảng điểm ảnh.

Ràng buộc và lưu ý

- Tên trong name là duy nhất; số ảnh và số người mỗi ảnh hữu hạn nhỏ, nhưng nên tiền xử lý Map vì nhiều ảnh dùng chung bảng điểm.

Contract

Tính tổng điểm của mỗi ảnh, tên không có trong bảng đóng góp 0.

Pattern và dấu hiệu nhận diện

Map lookup + reduce. Nhiều truy vấn ảnh dùng chung bảng name→yearning.

State • Transition • Invariant

Tiền xử lý Map; với mỗi photo reduce các tên. Invariant: sau k tên, sum là tổng đúng của prefix k.

Complexity

$O(\text{names} + \text{tổng số tên trong photo})$, $O(\text{names})$.

Bẫy / edge case

- Dùng truthy khiến điểm 0 bị hiểu là thiếu; trả sai thứ tự ảnh.

Code JavaScript

```
function solution(name, yearning, photo) {
  const score = new Map(name.map((x, i) => [x, yearning[i]]));
  return photo.map((pic) => pic.reduce((s, x) => s + (score.get(x) ?? 0), 0));
}
```

Bài 12 — Đi dạo công viên (공원 산책)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/172928?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Một công viên là lưới ký tự gồm S (điểm xuất phát), O (đi được) và X (vật cản). Mỗi route có dạng “hướng số_bước”. Chỉ thực hiện route nếu mọi ô đi qua đều nằm trong lưới và không phải X; nếu bất kỳ bước nào lỗi thì bỏ toàn bộ route và giữ nguyên vị trí trước lệnh. Sau tất cả route, trả hàng và cột cuối.

Input / Output

Nhận park và routes; hướng thuộc N,S,W,E. Trả [row,col] zero-based.

Ràng buộc và lưu ý

- Lưới tối đa 50×50; mỗi route đi tối đa 9 bước. Phải kiểm tra cả các ô trung gian, không chỉ ô đích.

Contract

Trả [row,col] sau khi bỏ qua mọi route đi ra biên hoặc xuyên vật cản.

Pattern và dấu hiệu nhận diện

Grid simulation + thử rồi commit. Lệnh chỉ hợp lệ nếu toàn bộ đường đi hợp lệ.

State • Transition • Invariant

Tìm S. Với mỗi route, copy state sang nr,nc và đi từng bước; chỉ gán lại r,c nếu không lỗi. Invariant: state thật luôn là kết quả sau các lệnh hợp lệ hoàn chỉnh.

Complexity

O(HW + tổng bước), O(1).

Bẫy / edge case

- Chỉ kiểm tra ô đích; di chuyển một phần rồi không rollback; đảo row/col.

Code JavaScript

```
function solution(park, routes) {
  const H = park.length,
        W = park[0].length,
        d = { N: [-1, 0], S: [1, 0], W: [0, -1], E: [0, 1] };
  let r = 0,
      c = 0;
  for (let i = 0; i < H; i++)
    for (let j = 0; j < W; j++) if (park[i][j] === 'S') [r, c] = [i, j];
  for (const s of routes) {
    const [o, z] = s.split(' '),
          [dr, dc] = d[o];
    let nr = r,
        nc = c,
        ok = true;
    for (let k = 0; k < +z; k++) {
      nr += dr;
      nc += dc;
      if (nr < 0 || nr >= H || nc < 0 || nc >= W || park[nr][nc] === 'X') {
        ok = false;
        break;
      }
    }
    if (ok) [r, c] = [nr, nc];
  }
  return [r, c];
}
```

Bài 13 — Dọn hình nền (바탕화면 정리)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/161990?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi dòng wallpaper biểu diễn màn hình: '#' là file, '.' là ô trống. Muốn kéo chuột một hình chữ nhật theo các đường biên ô để chọn tất cả file với quãng kéo Manhattan ngắn nhất. Hãy trả tọa độ góc trên-trái và dưới-phải của hình chữ nhật bao nhỏ nhất. Ô file (r,c) yêu cầu biên dưới/phải ít nhất là r+1,c+1.

Input / Output

Nhận wallpaper là mảng chuỗi cùng độ dài. Trả [lux,luy,rdx,rdy].

Ràng buộc và lưu ý

- Luôn có ít nhất một file. Tọa độ dưới-phải là biên exclusive, không phải index file lớn nhất.

Contract

Trả hình chữ nhật nhỏ nhất [lux,luy,rdx,rdy] chứa mọi ô '#'.
[lux, luy, rdx, rdy] = [row, col, row, col]

Pattern và dấu hiệu nhận diện

Bounding box trên matrix. Mỗi file cập nhật min row/col và max biên dưới/phải.

State • Transition • Invariant

Khởi tạo min=Infinity, max=-Infinity. Với file tại (r,c), cập nhật min bằng r,c và max bằng r+1,c+1.

Invariant: box bao đúng mọi file đã quét.

Complexity

O(HW), O(1).

Bẫy / edge case

- Trả max index thay vì biên exclusive +1; nhầm x/y với row/col.

Code JavaScript

```
function solution(wallpaper) {
  let a = Infinity,
      b = Infinity,
      c = -1,
      d = -1;
  wallpaper.forEach((row, i) => {
    [...row].forEach((x, j) => {
      if (x === '#') {
        a = Math.min(a, i);
        b = Math.min(b, j);
        c = Math.max(c, i + 1);
        d = Math.max(d, j + 1);
      }
    });
  });
  return [a, b, c, d];
}
```

Bài 14 — Sơn lại (뎃칠하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/161989?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Một bức tường dài n ô, con lăn phủ đúng m ô liên tiếp. section đã tăng dần chứa các ô phải sơn lại; được phép sơn cả ô không nằm trong section. Mỗi lượt đặt con lăn trong phạm vi tường và phủ một đoạn dài m . Hãy tìm số lượt ít nhất để mọi ô yêu cầu được phủ.

Input / Output

Nhận $n, m, \text{section}$. Trả số lượt sơn tối thiểu.

Ràng buộc và lưu ý

- $1 \leq m \leq n$; section không trùng và tăng dần. Khi gặp ô chưa phủ sớm nhất, đặt con lăn bắt đầu tại đó là lựa chọn greedy tối ưu.

Contract

Tìm số lượt lăn dài m ít nhất để phủ các section cần sơn.

Pattern và dấu hiệu nhận diện

Greedy interval covering. Gặp vị trí chưa phủ sớm nhất thì bắt đầu một lượt sơn tại đó là lựa chọn không thua phương án khác.

State • Transition • Invariant

end là vị trí cuối đã phủ. Duyệt section tăng dần; nếu $x > \text{end}$ thì sơn $[x, x+m-1]$, tăng answer. Invariant: prefix trước x đã phủ với số lượt tối thiểu.

Complexity

$O(\text{section.length})$, $O(1)$.

Bẫy / edge case

- Dùng $\text{end} = x + m$ gây dư một ô; sort lại không cần vì input đã tăng; đếm mọi section.

Code JavaScript

```
function solution(n, m, section) {
  let end = 0;
  answer = 0;
  for (const x of section)
    if (x > end) {
      answer++;
      end = x + m - 1;
    }
  return answer;
}
```

Bài 15 — Bàn phím được tạo đại khái (대충 만든 자판)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/160586?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi chuỗi trong keymap mô tả một phím: ký tự ở index 0 cần bấm 1 lần, index 1 cần 2 lần,... Một ký tự có thể xuất hiện trên nhiều phím; luôn chọn số lần bấm ít nhất. Với từng target, tính tổng số lần bấm để tạo toàn bộ chuỗi; nếu thiếu chỉ một ký tự thì kết quả target là -1.

Input / Output

Nhận keymap và targets. Trả mảng chi phí tương ứng từng target.

Ràng buộc và lưu ý

- Mỗi mảng và mỗi chuỗi tối đa khoảng 100 phần tử/ký tự. Chi phí là index+1; phải lấy min giữa mọi keymap.

Contract

Trả số lần bấm ít nhất cho từng target; thiếu bất kỳ ký tự nào thì -1.

Pattern và dấu hiệu nhận diện

Preprocessing Map ký tự→chi phí nhỏ nhất. Nhiều target dùng chung keymap.

State • Transition • Invariant

Quét keymap cập nhật minPress[ch]. Với target, cộng lookup và dừng khi thiếu. Invariant sau preprocess: Map là min toàn cục.

Complexity

O(tổng độ dài keymap + targets), O(alphabet).

Bẫy / edge case

- Nhầm index với số lần bấm; ghi đè mà không lấy min; tiếp tục cộng sau khi thiếu.

Code JavaScript

```
function solution(keymap, targets) {
  const cost = new Map();
  for (const key of keymap)
    for (let i = 0; i < key.length; i++)
      cost.set(key[i], Math.min(cost.get(key[i]) ?? Infinity, i + 1));
  return targets.map((s) => {
    let sum = 0;
    for (const ch of s) {
      if (!cost.has(ch)) return -1;
      sum += cost.get(ch);
    }
    return sum;
  });
}
```


Bài 16 — Chồng thẻ (카드 뭉치)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/159994?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Có hai chồng thẻ `cards1` và `cards2`; chỉ được lấy thẻ ở đầu mỗi chồng, không đổi thứ tự trong chồng. Hãy kiểm tra có thể tạo chuỗi goal theo thứ tự bằng cách ở mỗi bước lấy đầu của một trong hai chồng hay không. Không bắt buộc dùng hết thẻ sau khi tạo xong goal.

Input / Output

Nhận `cards1`, `cards2`, `goal`. Trả “Yes” nếu tạo được, ngược lại “No”.

Ràng buộc và lưu ý

- Các từ trong mỗi chồng không lặp theo điều kiện đề. Không được lấy một từ nằm sâu hơn khi chưa dùng các thẻ phía trước.

Contract

Kiểm tra goal có thể tạo bằng cách lần lượt lấy đầu `cards1` hoặc `cards2`.

Pattern và dấu hiệu nhận diện

Hai con trỏ trên hai queue cố định. Chỉ được lấy đầu mỗi pile nên không cần shift.

State • Transition • Invariant

Giữ `i, j`. Với mỗi word: ưu tiên nếu bằng `cards1[i]`, nếu không thử `cards2[j]`, không khớp thì No. Invariant: `i, j` là số thẻ đã dùng.

Complexity

$O(\text{goal.length})$, $O(1)$.

Bẫy / edge case

- Dùng includes bỏ qua thứ tự; shift gây mutate không cần; tăng sai con trỏ.

Code JavaScript

```
function solution(cards1, cards2, goal) {
  let i = 0,
      j = 0;
  for (const w of goal) {
    if (cards1[i] === w) i++;
    else if (cards2[j] === w) j++;
    else return 'No';
  }
  return 'Yes';
}
```

Bài 17 — Mật mã của hai người (둘만의 암호)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/155652?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Cần mã hóa chuỗi `s` bằng cách dịch mỗi chữ cái tiến index vị trí trong alphabet, nhưng mọi chữ thuộc `skip` bị loại khỏi alphabet và không được đi qua/đáp xuống. Khi vượt `z` thì quay lại `a`. Hãy trả chuỗi mã hóa.

Input / Output

Nhận `s`, `skip` và `index`. Trả chuỗi chữ thường đã dịch.

Ràng buộc và lưu ý

- Chỉ có `a-z`; các chữ trong `s` không thuộc `skip`. Modulo phải dùng số chữ còn lại sau khi loại `skip`, không cố định 26.

Contract

Dịch mỗi chữ tiến index bước, bỏ qua các chữ trong `skip` và quay vòng `a-z`.

Pattern và dấu hiệu nhận diện

Character mapping + modulo trên alphabet đã lọc. Tạo representation hợp lệ trước khi dịch.

State • Transition • Invariant

Tạo `allowed` gồm 26 chữ không bị `skip`; map mỗi ch sang vị trí rồi $(pos + index) \% allowed.length$.

Complexity

$O(26 + |s| \cdot 26)$ với `indexOf`; $O(26)$.

Bẫy / edge case

- Dịch trên alphabet đầy đủ rồi mới bỏ `skip`; modulo 26 thay vì số chữ còn lại.

Code JavaScript

```
function solution(s, skip, index) {
  const banned = new Set(skip),
    a = [...Array(26)]
    .map((_, i) => String.fromCharCode(97 + i))
    .filter((c) => !banned.has(c));
  return [...s].map((c) => a[(a.indexOf(c) + index) % a.length]).join('');
}
```

Bài 18 — Thời hạn lưu thông tin cá nhân (개인정보 수집 유효기간)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/150370?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mọi tháng trong lịch giả lập có đúng 28 ngày. terms cho biết mỗi loại điều khoản được lưu bao nhiêu tháng. Mỗi privacy gồm ngày thu thập và loại điều khoản. Đến today, hồ sơ phải xóa nếu thời hạn lưu đã kết thúc; tại đúng ngày hết hạn cũng được coi là hết hiệu lực. Trả số thứ tự 1-based của các hồ sơ phải xóa theo thứ tự tăng.

Input / Output

Nhận today dạng YYYY.MM.DD, terms dạng "A 6", privacies dạng "YYYY.MM.DD A". Trả mảng index.

Ràng buộc và lưu ý

- Không dùng lịch thật; quy đổi toàn bộ sang một trục ngày 28-ngày/tháng. Điều kiện hết hạn là $\text{collected} + \text{duration} \leq \text{today}$.

Contract

Trả index 1-based của hồ sơ đã hết hạn tại today.

Pattern và dấu hiệu nhận diện

Chuẩn hoá ngày giả lập → một số + Map term. Khi đề quy định tháng 28 ngày, không dùng Date thật.

State • Transition • Invariant

toDays ánh xạ y,m,d lên cùng trục; $\text{expiry} = \text{collected} + \text{months} \cdot 28$. Hết hạn khi $\text{expiry} \leq \text{today}$.

Complexity

$O(\text{terms} + \text{privacies})$, $O(\text{terms})$.

Bẫy / edge case

- Nhằm < với $\leq$; dùng lịch thật; quên index 1-based.

Code JavaScript

```
function solution(today, terms, privacies) {
  const day = (s) => {
    const [y, m, d] = s.split('.').map(Number);
    return (y * 12 + m) * 28 + d;
  };
  const term = new Map(
    terms.map((s) => {
      const [a, b] = s.split(' ');
      return [a, +b];
    })
  ),
  now = day(today),
  ans = [];
  privacies.forEach((s, i) => {
    const [d, t] = s.split(' ');
    if (day(d) + term.get(t) * 28 <= now) ans.push(i + 1);
  });
  return ans;
}
```

Bài 19 — Chuỗi con có giá trị nhỏ (크기가 작은 부분 문자열)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/147355?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Với chuỗi số t và chuỗi số p , xét mọi chuỗi con liên tiếp của t có cùng độ dài với p . Chuyển mỗi chuỗi con thành giá trị số và đếm bao nhiêu giá trị nhỏ hơn hoặc bằng p . Các số 0 ở đầu chuỗi con không làm thay đổi độ dài của số.

Input / Output

Nhận t và p là chuỗi chữ số. Trả số của số thỏa điều kiện.

Ràng buộc và lưu ý

- Độ dài t có thể tới 10.000, p tối đa 18 chữ số theo đề gốc; các cửa sổ có cùng độ dài nên có thể so sánh hoặc dùng Number trong bound được cung cấp.

Contract

Đếm substring của t dài $|p|$ có giá trị không lớn hơn p .

Pattern và dấu hiệu nhận diện

Fixed-size sliding window + so sánh số. Mọi candidate có cùng độ dài p .

State • Transition • Invariant

Duyệt start $0..|t|-|p|$; slice đúng độ dài và Number để so. Invariant: answer đếm đúng các cửa sổ trước start.

Complexity

$O(|t| \cdot |p|)$ do slice; bound Level 1 đủ nhanh, $O(|p|)$ tạm.

Bẫy / edge case

- Lềch cận cuối; so chuỗi khi có trường hợp cần số; dùng parseInt thiếu radix không cần thiết.

Code JavaScript

```
function solution(t, p) {
  let ans = 0;
  for (let i = 0; i + p.length <= t.length; i++)
    if (Number(t.slice(i, i + p.length)) <= Number(p)) ans++;
  return ans;
}
```

Bài 20 — Chữ giống gần nhất (가장 가까운 같은 글자)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/142086?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Duyệt chuỗi s từ trái sang phải. Với mỗi ký tự, nếu nó chưa từng xuất hiện trước đó thì ghi -1; nếu đã xuất hiện, ghi khoảng cách tới lần xuất hiện gần nhất bên trái. Sau khi xử lý ký tự hiện tại, vị trí này trở thành lần xuất hiện gần nhất mới.

Input / Output

Nhận s. Trả mảng số có cùng độ dài.

Ràng buộc và lưu ý

- Chỉ chữ thường; cần lưu vị trí cuối, không phải vị trí đầu. Ví dụ “banana” cho [-1,-1,-1,2,2,2].

Contract

Với mỗi ký tự, trả khoảng cách tới lần xuất hiện gần nhất trước đó hoặc -1.

Pattern và dấu hiệu nhận diện

Map last position. “Gần nhất bên trái” là lookup vị trí cuối đã thấy.

State • Transition • Invariant

last[ch] lưu index lớn nhất đã xử lý. Tại i, đọc last rồi cập nhật i. Invariant: Map chỉ chứa prefix bên trái.

Complexity

O(n), O(alphabet).

Bẫy / edge case

- Cập nhật trước khi tính khiến khoảng cách 0; dùng indexOf tìm lần đầu thay vì lần gần nhất.

Code JavaScript

```
function solution(s) {
  const last = new Map();
  return [...s].map((c, i) => {
    const v = last.has(c) ? i - last.get(c) : -1;
    last.set(c, i);
    return v;
  });
}
```

Bài 21 — Tách chuỗi (문자열 나누기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/140108?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Bắt đầu một đoạn mới tại ký tự x đầu tiên chưa dùng. Trong đoạn, đếm số ký tự bằng x và số ký tự khác x. Ngay khi hai số bằng nhau, kết thúc đoạn và tiếp tục với phần còn lại. Nếu hết chuỗi mà chưa cân bằng, phần dư vẫn tạo một đoạn. Hãy đếm tổng số đoạn.

Input / Output

Nhận chuỗi s. Trả số đoạn theo quy tắc.

Ràng buộc và lưu ý

- Không cần các đoạn có nội dung giống nhau; mỗi đoạn xác định greedy từ trái sang phải. Ký tự chuẩn x được reset khi bắt đầu đoạn mới.

Contract

Đếm số đoạn tạo theo quy tắc để khi quét trái→phải.

Pattern và dấu hiệu nhận diện

Greedy scan + bộ đếm cân bằng. Mỗi đoạn kết thúc sớm nhất khi số ký tự đầu bằng số ký tự khác.

State • Transition • Invariant

Khi bắt đầu đoạn, lưu first và reset same/diff. Sau mỗi ký tự, nếu same==diff thì chốt đoạn. Cuối chuỗi còn dở vẫn tính một đoạn.

Complexity

$O(n)$, $O(1)$.

Bẫy / edge case

- Quên đoạn cuối chưa cân bằng; đổi first giữa đoạn; so từng loại chữ thay vì “khác first”.

Code JavaScript

```
function solution(s) {
  let ans = 0,
      first = '',
      same = 0,
      diff = 0;
  for (const c of s) {
    if (same === diff) {
      first = c;
      same = 0;
      diff = 0;
      ans++;
    }
    c === first ? same++ : diff++;
  }
  return ans;
}
```

Bài 22 — Hall of Fame (1) (명예의 전당 (1))

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/138477?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi ngày có một score mới. Hall of Fame giữ tối đa k điểm cao nhất đã xuất hiện tính tới ngày đó. Sau khi cập nhật, công bố điểm nhỏ nhất đang nằm trong Hall of Fame. Trong k ngày đầu, Hall có ít hơn k người nhưng vẫn công bố min hiện tại. Hãy trả chuỗi điểm công bố.

Input / Output

Nhận k và score. Trả mảng min theo ngày.

Ràng buộc và lưu ý

- $1 \leq k \leq 100$; score dài tối đa khoảng 1.000. Chỉ cần duy trì top-k, không phải min của toàn bộ lịch sử.

Contract

Trả điểm thấp nhất trong Hall of Fame sau từng score.

Pattern và dấu hiệu nhận diện

Duy trì top-k nhỏ bằng sort. Sau mỗi ngày cần min của k điểm cao nhất.

State • Transition • Invariant

Push score, sort giảm, cắt còn k; phần tử cuối là ngưỡng. Invariant: hall đúng k điểm lớn nhất của prefix.

Complexity

$O(n \cdot k \log k)$, $O(k)$; k, n nhỏ. Có thể heap nhưng không cần.

Bẫy / edge case

- Lấy min toàn bộ lịch sử; sort tăng nhưng đọc sai đầu/cuối; giữ hơn k phần tử.

Code JavaScript

```
function solution(k, score) {
  const hall = [];
  return score.map((x) => {
    hall.push(x);
    hall.sort((a, b) => b - a);
    if (hall.length > k) hall.pop();
    return hall[hall.length - 1];
  });
}
```

Bài 23 — Vũ khí hiệp sĩ (기사단원의 무기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/136798?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Các hiệp sĩ mang số 1..number. Sức mạnh vũ khí của hiệp sĩ x bằng số ước dương của x. Nếu sức mạnh này vượt limit, thay bằng power; nếu bằng limit thì vẫn giữ nguyên. Tổng nguyên liệu cần dùng bằng tổng sức mạnh cuối của mọi vũ khí. Hãy tính tổng đó.

Input / Output

Nhận number, limit, power. Trả tổng nguyên liệu.

Ràng buộc và lưu ý

- Có thể đếm ước theo cặp đến căn bậc hai. Với số chính phương, căn chỉ được đếm một lần.

Contract

Tổng sức mạnh vũ khí cho 1..number; nếu số ước vượt limit thì dùng power.

Pattern và dấu hiệu nhận diện

Đếm ước bằng cặp tới sqrt + threshold substitution.

State • Transition • Invariant

Với mỗi x, duyệt $d \leq \sqrt{x}$; cộng 2 cho cặp, 1 nếu bình phương. Invariant sau d: đã đếm mọi ước có phần tử nhỏ $\leq d$.

Complexity

$O(\text{number} \cdot \sqrt{\text{number}})$, $O(1)$.

Bẫy / edge case

- Đếm số chính phương hai lần; thay khi $\geq \text{limit}$ thay vì $> \text{limit}$; tối ưu sớm sai.

Code JavaScript

```
function solution(number, limit, power) {
    let sum = 0;
    for (let x = 1; x <= number; x++) {
        let c = 0;
        for (let d = 1; d * d <= x; d++) if (x % d === 0) c += d * d === x ? 1 : 2;
        sum += c > limit ? power : c;
    }
    return sum;
}
```


Bài 24 — Người bán trái cây (과일 장수)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/135808?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi quả có điểm chất lượng. Một hộp phải có đúng m quả; giá bán hộp bằng điểm nhỏ nhất trong hộp nhân m . Có thể bỏ quả dư và mỗi quả chỉ dùng một lần. Hãy chia hộp để tối đa tổng tiền.

Input / Output

Nhận k (điểm tối đa không ảnh hưởng công thức), m và $score$. Trả lợi nhuận lớn nhất.

Ràng buộc và lưu ý

- Sort giảm rồi gom từng nhóm m ; phần tử cuối nhóm là $\min$. Không tạo hộp thiếu m quả.

Contract

Tối đa hoá lợi nhuận khi đóng các hộp đủ k quả.

Pattern và dấu hiệu nhận diện

Sort giảm + greedy chia nhóm. Trong mỗi hộp k quả, giá trị do quả nhỏ nhất quyết định.

State • Transition • Invariant

Sort giảm; mỗi block k lấy phần tử cuối block làm $\min$, cộng $\min \cdot k$. Ghép các quả lớn cùng nhau tránh kéo thấp nhiều hộp.

Complexity

$O(n \log n)$, $O(n)$ hoặc $O(1)$ ngoài sort mutate.

Bẫy / edge case

- Tính theo max hộp; dùng phần dư thành hộp thiếu; quên sort số.

Code JavaScript

```
function solution(k, m, score) {
  score.sort((a, b) => b - a);
  let sum = 0;
  for (let i = k - 1; i < score.length; i += k) sum += score[i] * k;
  return sum;
}
```

Bài 25 — Cuộc thi Food Fight (푸드 파이트 대회)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/134240?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

food[i] là số món loại i, trong đó food[0] là nước. Hai người cần nhận lượng và thứ tự món đối xứng, ăn từ ngoài vào giữa; nước “0” đặt chính giữa. Với mỗi loại $i \geq 1$, chỉ dùng một số chẵn lớn nhất không vượt food[i], chia đôi cho hai phía. Tạo chuỗi biểu diễn cách xếp.

Input / Output

Nhận food. Trả chuỗi gồm chỉ số món và một ký tự 0 ở giữa.

Ràng buộc và lưu ý

- Bỏ một món nếu số lượng lẻ. Nửa phải là đảo của nửa trái, không phải copy cùng chiều.

Contract

Tạo cách xếp đối xứng quanh nước 0.

Pattern và dấu hiệu nhận diện

Xây nửa chuỗi + mirror. Mỗi loại dùng $\text{floor}(\text{food}[i]/2)$ ở mỗi phía.

State • Transition • Invariant

Duyệt loại 1.., nối i lặp $\text{floor}(\text{count}/2)$ vào left; đáp án $\text{left} + '0' + \text{reverse}(\text{left})$.

Complexity

O(tổng số món được dùng), O(output).

Bẫy / edge case

- Dùng cả số lẻ; reverse chuỗi sai cách; lặp food[0].

Code JavaScript

```
function solution(food) {
  let left = '';
  for (let i = 1; i < food.length; i++)
    left += String(i).repeat(Math.floor(food[i] / 2));
  return left + '0' + [...left].reverse().join('');
}
```

Bài 26 — Làm hamburger (햄버거 만들기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/133502?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Một băng chuyền đưa nguyên liệu theo thứ tự. Một hamburger hoàn chỉnh là chuỗi liên tiếp 1-2-3-1. Khi mẫu xuất hiện ở phần đã nhận, đóng gói và loại bốn nguyên liệu đó; việc loại có thể làm lộ ra mẫu mới ở ranh giới. Hãy đếm số hamburger đóng được sau khi xử lý toàn bộ.

Input / Output

Nhận ingredient là mảng số 1,2,3. Trả số hamburger.

Ràng buộc và lưu ý

- Độ dài có thể tới 1.000.000; replace chuỗi lặp để $O(n^2)$. Stack mô tả chính xác phần nguyên liệu còn lại.

Contract

Đếm số lần xoá tuần tự pattern 1,2,3,1.

Pattern và dấu hiệu nhận diện

Stack + suffix pattern removal. Mẫu chỉ có thể xuất hiện mới ở cuối prefix vừa push.

State • Transition • Invariant

Push từng nguyên liệu; nếu 4 phần tử cuối khớp, giảm length 4 và tăng answer. Invariant: stack là prefix sau mọi xoá có thể thực hiện.

Complexity

$O(n)$, $O(n)$.

Bẫy / edge case

- Dùng join/replace lặp thành $O(n^2)$; kiểm tra pattern trước push; xoá sai thứ tự.

Code JavaScript

```
function solution(ingredient) {
  const st = [];
  let ans = 0;
  for (const x of ingredient) {
    st.push(x);
    const n = st.length;
    if (
      n >= 4 &&
      st[n - 4] === 1 &&
      st[n - 3] === 2 &&
      st[n - 2] === 3 &&
      st[n - 1] === 1
    ) {
      st.length -= 4;
      ans++;
    }
  }
  return ans;
}
```

Bài 27 — Bập bẹ (2) (옹알이 (2))

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/133499?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Một từ hợp lệ nếu ghép hoàn toàn từ bốn âm “aya”, “ye”, “woo”, “ma”, nhưng không được dùng cùng một âm hai lần liên tiếp. Các âm có thể lặp lại nếu có âm khác xen giữa. Hãy đếm số từ trong babbling hợp lệ.

Input / Output

Nhận mảng babbling. Trả số từ phát âm được.

Ràng buộc và lưu ý

- Mỗi chuỗi phải được tiêu thụ hết; không chấp nhận chỉ prefix hợp lệ. Cấm lặp theo token, không phải cấm ký tự trùng.

Contract

Đếm word có thể phân rã hoàn toàn thành aya, ye, woo, ma mà không lặp token kề nhau.

Pattern và dấu hiệu nhận diện

Tokenization hữu hạn + kiểm tra token liên tiếp. Chỉ bốn âm hợp lệ và không được lặp ngay.

State • Transition • Invariant

Tại index, thử các token; token trùng prev không được chọn. Nếu không token nào khớp thì invalid; hết chuỗi thì valid.

Complexity

O(tổng độ dài·4), O(1).

Bẫy / edge case

- Regex thay rỗng nhưng bỏ sót cấm lặp; cho phép prefix hợp lệ nhưng còn ký tự thừa.

Code JavaScript

```
function solution(babbling) {
    const words = ['aya', 'ye', 'woo', 'ma'];
    let ans = 0;
    for (const s of babbling) {
        let i = 0,
            prev = '',
            ok = true;
        while (i < s.length) {
            const w = words.find((x) => x !== prev && s.startsWith(x, i));
            if (!w) {
                ok = false;
                break;
            }
            prev = w;
            i += w.length;
        }
        if (ok) ans++;
    }
    return ans;
}
```

Bài 28 — Bài toán cola (콜라 문제)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/132267?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Có n chai rỗng. Mỗi lần đưa a chai rỗng sẽ nhận b chai cola mới; uống cola mới tạo ra b chai rỗng tương ứng. Có thể đổi nhiều nhóm a trong một lượt và giữ lại số chai dư cho vòng sau. Hãy tính tổng số chai cola nhận được cho tới khi không đủ a chai để đổi.

Input / Output

Nhận a, b, n . Trả tổng chai cola mới.

Ràng buộc và lưu ý

- Điều kiện đảm bảo $a > b$ nên quá trình kết thúc. State vòng sau là $n \% a$ cộng số chai vừa nhận, không được bỏ phần dư.

Contract

Tính tổng chai mới nhận khi đổi a vỏ lấy b chai cho tới khi không đủ.

Pattern và dấu hiệu nhận diện

Simulation đổi vỏ + giữ phần dư. State là số chai rỗng đang có.

State • Transition • Invariant

Mỗi vòng $q = \text{floor}(n/a)$; nhận $q \cdot b$, phần vỏ mới là $n \% a + q \cdot b$. Invariant: n là tổng vỏ có thể dùng ở đầu vòng.

Complexity

$O(\log n)$ theo số vòng, $O(1)$.

Bẫy / edge case

- Bỏ phần dư; cộng số lượt q thay vì chai $q \cdot b$; vòng lặp khi $n < a$.

Code JavaScript

```
function solution(a, b, n) {
  let ans = 0;
  while (n >= a) {
    const q = Math.floor(n / a),
    got = q * b;
    ans += got;
    n = (n % a) + got;
  }
  return ans;
}
```

Bài 29 — Bộ ba số 0 (삼총사)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/131705?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Chọn ba học sinh có chỉ số khác nhau. Một bộ được gọi là bộ ba nếu tổng ba số trong number bằng 0. Hãy đếm số tổ hợp index thỏa mãn; các giá trị có thể trùng nhưng các vị trí khác nhau vẫn tạo các tổ hợp riêng.

Input / Output

Nhận number. Trả số bộ ba.

Ràng buộc và lưu ý

- Độ dài nhỏ, cho phép ba vòng lặp. Duyệt $i < j < k$ để mỗi tổ hợp được đếm đúng một lần.

Contract

Đếm bộ ba index khác nhau có tổng bằng 0.

Pattern và dấu hiệu nhận diện

Brute force chọn 3. Bound nhỏ và số tổ hợp hữu hạn.

State • Transition • Invariant

Ba vòng $i < j < k$ đảm bảo mỗi tổ hợp đúng một lần. Invariant: sau các vòng, mọi bộ ba theo thứ tự từ điển trước vị trí hiện tại đã xét.

Complexity

$O(n^3)$, $O(1)$.

Bẫy / edge case

- Dùng Set làm mất duplicate index; dùng permutation đếm 6 lần; cho phép lặp cùng index.

Code JavaScript

```
function solution(number) {
    let ans = 0;
    for (let i = 0; i < number.length; i++)
        for (let j = i + 1; j < number.length; j++)
            for (let k = j + 1; k < number.length; k++)
                if (number[i] + number[j] + number[k] === 0) ans++;
    return ans;
}
```

Bài 30 — Cặp số (숫자 짝꿍)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/131128?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Cho hai số rất lớn dưới dạng chuỗi X,Y. Có thể lấy mỗi chữ số ở mỗi chuỗi không quá số lần nó xuất hiện để tạo một “số chung”. Hãy tạo số chung lớn nhất bằng cách dùng số lần min cho từng digit rồi sắp giảm dần. Nếu không có digit chung trả “-1”; nếu kết quả chỉ gồm số 0 thì trả “0”.

Input / Output

Nhận X,Y là chuỗi chữ số. Trả chuỗi đáp án, không chuyển toàn bộ sang Number.

Ràng buộc và lưu ý

- Chuỗi có thể dài tới hàng triệu; chỉ cần hai mảng tần suất 10 phần tử. Phải chuẩn hóa nhiều số 0 thành một “0”.

Contract

Tạo số lớn nhất dùng các chữ số xuất hiện ở cả X và Y với số lần min.

Pattern và dấu hiệu nhận diện

Frequency counting trên 10 chữ số + dựng đáp án giảm dần.

State • Transition • Invariant

Đếm 0..9 cho hai chuỗi; nối từ 9 xuống 0 theo min count. Nếu rỗng trả -1; nếu ký tự đầu 0 thì trả 0.

Complexity

$O(|X|+|Y|)$, $O(10)$ ngoài output.

Bẫy / edge case

- Sort hàng triệu ký tự không cần; trả nhiều số 0; dùng Number làm mất độ dài và có thể overflow.

Code JavaScript

```
function solution(X, Y) {
  const a = Array(10).fill(0),
        b = Array(10).fill(0);
  for (const c of X) a[+c]++;
  for (const c of Y) b[+c]++;
  let s = '';
  for (let d = 9; d >= 0; d--) s += String(d).repeat(Math.min(a[d], b[d]));
  return s === '' ? '-1' : s[0] === '0' ? '0' : s;
}
```

Bài 31 — Trắc nghiệm tính cách (성격 유형 검사하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/118666?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi câu survey[i] gồm hai loại tính cách đối nghịch. choice 1..3 cho điểm loại bên trái lần lượt 3,2,1; choice 4 không cho điểm; choice 5..7 cho loại bên phải 1,2,3. Sau mọi câu, với từng cặp RT, CF, JM, AN chọn loại có điểm cao hơn; nếu hòa chọn chữ đứng trước theo alphabet. Ghép bốn lựa chọn thành kết quả.

Input / Output

Nhận survey và choices song song. Trả chuỗi 4 ký tự.

Ràng buộc và lưu ý

- Tối đa 1.000 câu. Quy tắc hòa phải được mã hóa rõ; choice=4 không cập nhật điểm.

Contract

Tính bốn ký tự đại diện từ survey và choices.

Pattern và dấu hiệu nhận diện

Counting + tie-break cố định. Dấu $\geq$ mã hoá quy tắc hòa chọn chữ trước.

State • Transition • Invariant

choice<4 cộng 4-choice cho bên trái; >4 cộng choice-4 cho bên phải. Với RT,CF,JM,AN chọn a nếu score[a] $\geq$ score[b].

Complexity

O(n), O(1).

Bẫy / edge case

- Cộng điểm choice=4; đảo trái/phải; dùng > làm sai tie-break.

Code JavaScript

```
function solution(survey, choices) {
  const sc = new Map([...'RTCFJMAN'].map((c) => [c, 0]));
  survey.forEach((s, i) => {
    const c = choices[i];
    if (c < 4) sc.set(s[0], sc.get(s[0]) + 4 - c);
    else if (c > 4) sc.set(s[1], sc.get(s[1]) + c - 4);
  });
  return ['RT', 'CF', 'JM', 'AN']
    .map(([a, b]) => (sc.get(a) >= sc.get(b) ? a : b))
    .join('');
}
```


Bài 32 — Nhận kết quả báo cáo (신고 결과 받기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/92334?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi report có dạng “người_báo người_bị_báo”. Một người chỉ có thể báo cùng mục tiêu một lần có hiệu lực, dù dòng report bị lặp. Người bị ít nhất k người khác nhau báo sẽ bị đình chỉ. Mọi người từng báo một tài khoản bị đình chỉ nhận một email. Trả số email của từng user theo thứ tự id_list.

Input / Output

Nhận id_list, report, k. Trả mảng số email.

Ràng buộc và lưu ý

- Tối đa 1.000 user và 200.000 report; phải khử trùng cặp báo cáo. Một reporter có thể nhận nhiều email nếu nhiều mục tiêu bị đình chỉ.

Contract

Trả số email mỗi user nhận sau khi đình chỉ người bị ít nhất k user khác báo.

Pattern và dấu hiệu nhận diện

Set khử trùng + Map quan hệ. “Cùng một cặp chỉ tính một lần” là tín hiệu Set.

State • Transition • Invariant

Khử trùng report; reportedBy[target] là Set reporter. Với mỗi target đủ k, tăng mail cho mọi reporter. Trả theo id_list.

Complexity

O(id_list+report duy nhất), O(report duy nhất).

Bẫy / edge case

- Đếm report trùng; trả theo thứ tự Map; đếm số target bị đình chỉ thay vì email.

Code JavaScript

```
function solution(id_list, report, k) {
  const by = new Map(id_list.map((x) => [x, new Set()]));
  for (const s of new Set(report)) {
    const [a, b] = s.split(' ');
    by.get(b).add(a);
  }
  const mail = new Map(id_list.map((x) => [x, 0]));
  for (const rs of by.values())
    if (rs.size >= k) for (const x of rs) mail.set(x, mail.get(x) + 1);
  return id_list.map((x) => mail.get(x));
}
```

Bài 33 — Tìm số có dư 1 (나머지가 1이 되는 수 찾기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/87389?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tìm số tự nhiên x nhỏ nhất sao cho khi chia n cho x thì dư đúng 1.

Input / Output

Nhận n. Trả x nhỏ nhất.

Ràng buộc và lưu ý

- $3 \leq n \leq 1.000.000$. Duyệt x tăng từ 2 và trả ngay lần đầu đúng là đủ; x=1 không thể cho dư 1.

Contract

Tìm x nhỏ nhất sao cho $n \% x = 1$.

Pattern và dấu hiệu nhận diện

Linear search theo định nghĩa. Cần giá trị nhỏ nhất nên duyệt tăng và trả ngay.

State • Transition • Invariant

Duyệt x từ 2; điều kiện đầu tiên đúng là đáp án do thứ tự tăng. Không cần factorization phức tạp.

Complexity

O(n) worst-case, O(1).

Bẫy / edge case

- Bắt đầu từ 1; dùng n/x; tiếp tục sau khi đã tìm min.

Code JavaScript

```
function solution(n) {  
  for (let x = 2; x < n; x++) if (n % x === 1) return x;  
}
```

Bài 34 — Hình chữ nhật nhỏ nhất (최소직사각형)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/86491?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi tấm danh thiếp có kích thước $[w, h]$ và được phép xoay 90 độ độc lập. Cần làm một chiếc ví hình chữ nhật chứa được mọi tấm. Hãy chọn hướng từng tấm và trả diện tích ví nhỏ nhất.

Input / Output

Nhận sizes. Trả diện tích số nguyên.

Ràng buộc và lưu ý

- Chuẩn hóa mỗi tấm thành cạnh dài và cạnh ngắn; ví cần max của tất cả cạnh dài nhân max của tất cả cạnh ngắn.

Contract

Tìm diện tích ví nhỏ nhất chứa mọi card.

Pattern và dấu hiệu nhận diện

Normalize orientation + scan max. Mỗi card có thể xoay độc lập.

State • Transition • Invariant

Với mỗi $[a, b]$, đặt long=max, short=min; cập nhật maxLong, maxShort. Invariant: hai max là kích thước cần cho prefix đã đọc.

Complexity

$O(n)$, $O(1)$.

Bẫy / edge case

- Chọn orientation toàn cục thay vì từng card; nhân max của cột gốc.

Code JavaScript

```
function solution(sizes) {
  let a = 0,
      b = 0;
  for (const [x, y] of sizes) {
    a = Math.max(a, x, y);
    b = Math.max(b, Math.min(x, y));
  }
  return a * b;
}
```

Bài 35 — Cộng số còn thiếu (없는 숫자 더하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/86051?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Trong các chữ số 0..9, numbers chứa một số chữ số không lặp. Hãy cộng tất cả chữ số không xuất hiện.

Input / Output

Nhận numbers. Trả tổng.

Ràng buộc và lưu ý

- Có thể dùng tổng đầy đủ 45 trừ tổng input. Không có duplicate theo đề.

Contract

Tính tổng các chữ số 0-9 không có trong numbers.

Pattern và dấu hiệu nhận diện

Set/complement hoặc công thức tổng 0..9.

State • Transition • Invariant

Tổng đầy đủ là 45; trừ tổng input. Duplicate không có theo đề, nên state chỉ là running sum.

Complexity

$O(n)$, $O(1)$.

Bẫy / edge case

- Dùng Set dù không sai nhưng quên constraint không trùng; tổng 1..9 cũng vẫn là 45 nhưng diễn giải phải đúng.

Code JavaScript

```
function solution(numbers) {  
    return 45 - numbers.reduce((a, b) => a + b, 0);  
}
```

Bài 36 — Tính số tiền thiếu (부족한 금액 계산하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/82612?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Một lượt chơi thứ i có giá $\text{price} \times i$. Người chơi có money và muốn chơi count lượt từ 1 tới count . Hãy tính số tiền còn thiếu; nếu đủ tiền thì trả 0.

Input / Output

Nhận $\text{price}, \text{money}, \text{count}$. Trả $\max(0, \text{tổng chi phí} - \text{money})$.

Ràng buộc và lưu ý

- Dùng tổng cấp số cộng $\text{price} \times \text{count} \times (\text{count} + 1) / 2$; các bound chính thức giữ kết quả trong miền số an toàn cần thiết.

Contract

Trả số tiền còn thiếu sau count lượt có giá $\text{price} \cdot i$.

Pattern và dấu hiệu nhận diện

Công thức cấp số cộng + clamp 0.

State • Transition • Invariant

$\text{need} = \text{price} \cdot \text{count} \cdot (\text{count} + 1) / 2 - \text{money}$; trả $\max(0, \text{need})$. Kiểm tra bound để Number còn an toàn.

Complexity

$O(1)$, $O(1)$.

Bẫy / edge case

- Quên không âm; off-by-one từ $i=0$; overflow trong ngôn ngữ số nguyên hẹp.

Code JavaScript

```
function solution(price, money, count) {  
    return Math.max(0, (price * count * (count + 1)) / 2 - money);  
}
```

Bài 37 — Chuỗi số và từ tiếng Anh (숫자 문자열과 영단어)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/81301?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Chuỗi s biểu diễn một số, trong đó một số chữ số có thể được viết bằng từ tiếng Anh zero, one,...,nine và phần còn lại là ký tự số. Các token nối liền không có khoảng trắng. Hãy thay đúng mọi tên số và trả giá trị số nguyên. Ví dụ “one4seveneight” thành 1478.

Input / Output

Nhận s . Trả số nguyên được mô tả.

Ràng buộc và lưu ý

- Mọi chuỗi đầu vào đều tạo thành một số hợp lệ; cần thay tất cả lần xuất hiện, không chỉ lần đầu.

Contract

Chuyển chuỗi chứa chữ số và tên số tiếng Anh thành số.

Pattern và dấu hiệu nhận diện

Multi-token replacement / parsing. Tập token cố định 10 phần tử.

State • Transition • Invariant

Thay mọi tên zero..nine bằng digit; vì token không gây nhập nhằng, replaceAll tuần tự là an toàn.

Complexity

$O(10 \cdot |s|)$, $O(|s|)$.

Bẫy / edge case

- replace chỉ lần đầu; parse trước khi thay xong; typo tên số.

Code JavaScript

```
function solution(s) {
  [
    'zero',
    'one',
    'two',
    'three',
    'four',
    'five',
    'six',
    'seven',
    'eight',
    'nine'
  ].forEach((w, i) => (s = s.replaceAll(w, i)));
  return Number(s);
}
```

Bài 38 — Số lượng ước và phép cộng (약수의 개수와 덧셈)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/77884?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Với mỗi số trong đoạn inclusive $[left, right]$, nếu số lượng ước của nó là chẵn thì cộng số đó vào tổng, nếu lẻ thì trừ. Hãy trả tổng cuối.

Input / Output

Nhận $left, right$. Trả tổng có dấu.

Ràng buộc và lưu ý

- Một số có lượng ước lẻ khi và chỉ khi là số chính phương; có thể dùng tính chất này thay vì đếm từng ước.

Contract

Cộng x nếu số ước chẵn, trừ nếu lẻ trên $[left, right]$.

Pattern và dấu hiệu nhận diện

Tính chất số chính phương. Số có số ước lẻ khi và chỉ khi là chính phương.

State • Transition • Invariant

Duyệt x ; `Number.isInteger(sqrt(x))` quyết định dấu. Đây là rút gọn của đếm ước.

Complexity

$O(right-left+1)$, $O(1)$.

Bẫy / edge case

- Dùng làm tròn `sqrt`; đảo dấu; bỏ hai đầu đoạn.

Code JavaScript

```
function solution(left, right) {  
  let sum = 0;  
  for (let x = left; x <= right; x++) sum += Number.isInteger(Math.sqrt(x)) ? -x : x;  
  return sum;  
}
```

Bài 39 — Hạng cao nhất và thấp nhất Lotto (로또의 최고 순위와 최저 순위)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/77484?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Một vé lotto có 6 số; số 0 là số bị mờ chưa biết và có thể trở thành bất kỳ số nào. win_nums là 6 số trúng. Tính hạng tốt nhất khi mọi số 0 được chọn thuận lợi nhất và hạng xấu nhất khi không số 0 nào tạo thêm match. Hạng theo số match: 6→1, 5→2,..., 2→5, còn 0 hoặc 1→6.

Input / Output

Nhận lottos và win_nums. Trả [bestRank, worstRank].

Ràng buộc và lưu ý

- Mỗi mảng dài 6; số khác 0 không trùng trong từng mảng. Đếm số chắc chắn match và số lượng 0.

Contract

Trả [best, worst] từ số 0 chưa biết và số chắc chắn khớp.

Pattern và dấu hiệu nhận diện

Counting matches + unknown range.

State • Transition • Invariant

matches đếm số có trong win_nums, zeros đếm 0. rank(k)=min(6, 7-k). best dùng matches+zeros, worst dùng matches.

Complexity

$O(6^2)$ hoặc $O(6)$ với Set, $O(6)$.

Bẫy / edge case

- Rank 0 match phải là 6; tính 0 như không khớp ở best; duplicate không có theo đề.

Code JavaScript

```
function solution(lottos, win_nums) {
  const win = new Set(win_nums),
    z = lottos.filter(x => x === 0).length,
    m = lottos.filter(x => win.has(x)).length,
    rank = (x) => Math.min(6, 7 - x);
  return [rank(m + z), rank(m)];
}
```


Bài 40 — Cộng âm dương (음양 더하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/76501?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

absolutes chứa trị tuyệt đối của các số; signs[i]=true nghĩa số thứ i dương, false nghĩa âm. Khôi phục dấu và tính tổng.

Input / Output

Nhận absolutes, signs cùng độ dài. Trả tổng số nguyên.

Ràng buộc và lưu ý

- Duyệt song song đúng index; true là cộng, false là trừ.

Contract

Tính tổng absolutes với signs true là dương, false là âm.

Pattern và dấu hiệu nhận diện

Zip arrays + reduce có dấu.

State • Transition • Invariant

Duyệt cùng index; cộng signs[i]?a:-a. Invariant: sum đúng cho prefix.

Complexity

O(n), O(1).

Bẫy / edge case

- Đảo nghĩa boolean; arrays lệch index; mutate input không cần.

Code JavaScript

```
function solution(absolutes, signs) {  
    return absolutes.reduce((s, x, i) => s + (signs[i] ? x : -x), 0);  
}
```

Bài 41 — Gợi ý ID mới (신규 아이디 추천)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/72410?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Chuẩn hóa new_id qua bảy bước theo thứ tự: đổi chữ hoa thành thường; xóa ký tự ngoài chữ thường, số, '-', '_', '.'; co nhiều dấu chấm liên tiếp thành một; xóa chấm đầu/cuối; nếu rỗng dùng "a"; nếu dài hơn 15 thì cắt 15 và xóa chấm cuối; nếu ngắn hơn 3 thì lặp ký tự cuối tới đủ 3. Trả ID cuối.

Input / Output

Nhận new_id. Trả chuỗi đã chuẩn hóa.

Ràng buộc và lưu ý

- Thứ tự bước là một phần của đề; đặc biệt phải trim chấm lại sau khi cắt 15. Độ dài đầu vào hữu hạn và chỉ gồm các ký tự được mô tả trong đề.

Contract

Chuẩn hoá new_id qua đúng 7 quy tắc và trả ID hợp lệ.

Pattern và dấu hiệu nhận diện

String normalization pipeline. Mỗi bước tương ứng một luật và output bước trước là input bước sau.

State • Transition • Invariant

Lowercase; lọc ký tự; co dấu chấm; trim chấm; fallback a; cắt 15 rồi trim; pad tới 3. Giữ pipeline theo thứ tự để.

Complexity

O(n), O(n).

Bẫy / edge case

- Đổi thứ tự cắt/trim; regex dấu chấm thiếu escape; pad khi chuỗi rỗng trước fallback.

Code JavaScript

```
function solution(id) {
  id = id
    .toLowerCase()
    .replace(/^[a-z0-9-_.]/g, '')
    .replace(/\.+/g, '.')
    .replace(/^\.|\.$/g, '');
  if (!id) id = 'a';
  id = id.slice(0, 15).replace(/\.$/, '');
  while (id.length < 3) id += id[id.length - 1];
  return id;
}
```

Bài 42 — Tích vô hướng (내적)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/70128?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tích vô hướng của hai vector a, b cùng chiều dài là tổng $a[i] \times b[i]$. Hãy tính giá trị này.

Input / Output

Nhận hai mảng số nguyên a, b . Trả một số nguyên.

Ràng buộc và lưu ý

- Hai mảng cùng độ dài; ghép phần tử theo cùng index, không phải mọi cặp.

Contract

Tính tổng $a[i] \cdot b[i]$.

Pattern và dấu hiệu nhận diện

Zip arrays + reduce.

State • Transition • Invariant

Accumulator sau i là dot product của prefix $[0..i]$.

Complexity

$O(n)$, $O(1)$.

Bẫy / edge case

- Nhân sai index; khởi tạo reduce thiếu 0 với input rỗng dù để có phần tử.

Code JavaScript

```
function solution(a, b) {  
  return a.reduce((s, x, i) => s + x * b[i], 0);  
}
```

Bài 43 — Đảo số hệ tam phân (3진법 뒤집기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/68935?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Viết n ở hệ cơ số 3, đảo thứ tự các chữ số của biểu diễn đó, rồi hiểu chuỗi đảo vẫn ở cơ số 3 và chuyển về thập phân. Hãy trả kết quả.

Input / Output

Nhận số tự nhiên n . Trả số thập phân sau biến đổi.

Ràng buộc và lưu ý

- Số 0 ở đầu chuỗi sau đảo có thể bị bỏ khi parse; điều đó không thay đổi giá trị.

Contract

Đổi n sang hệ 3, đảo chuỗi chữ số rồi đọc lại về hệ 10.

Pattern và dấu hiệu nhận diện

Base conversion + reverse representation.

State • Transition • Invariant

`toString(3)` tạo representation; `reverse`; `parseInt(base 3)`. Đây là pipeline biểu diễn, không cần mô phỏng chia tay.

Complexity

$O(\log n)$, $O(\log n)$.

Bẫy / edge case

- Quên radix 3 khi parse; đảo giá trị thập phân; mất digit 0 đầu là hợp lệ khi parse.

Code JavaScript

```
function solution(n) {  
  return parseInt([...n.toString(3)].reverse().join(''), 3);  
}
```

Bài 44 — Chọn hai số rồi cộng (두 개 뽑아서 더하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/68644?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Chọn hai phần tử ở hai index khác nhau trong numbers và tính tổng. Thu thập mọi tổng có thể, loại giá trị trùng và sắp tăng dần.

Input / Output

Nhận numbers. Trả mảng tổng duy nhất tăng dần.

Ràng buộc và lưu ý

- Độ dài nhỏ cho phép $O(n^2)$. Phải sort bằng comparator số trong JavaScript.

Contract

Trả các tổng của mọi cặp index khác nhau, duy nhất và tăng dần.

Pattern và dấu hiệu nhận diện

Brute force cặp + Set khử trùng + sort.

State • Transition • Invariant

Duyệt $i < j$; add sum vào Set; chuyển array và numeric sort. Invariant: Set chứa mọi tổng của cặp đã xét.

Complexity

$O(n^2 + u \log u)$, $O(u)$.

Bẫy / edge case

- Dùng sort() mặc định; cho $i=j$; quên unique.

Code JavaScript

```
function solution(numbers) {
  const s = new Set();
  for (let i = 0; i < numbers.length; i++)
    for (let j = i + 1; j < numbers.length; j++) s.add(numbers[i] + numbers[j]);
  return [...s].sort((a, b) => a - b);
}
```

Bài 45 — Bấm bàn phím số (키패드 누르기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/67256?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Bàn phím số có tọa độ chuẩn 1..9 và hàng cuối *,0,#. Ngón trái bắt đầu ở *, phải ở #. Phím cột trái 1,4,7 luôn dùng trái; cột phải 3,6,9 luôn dùng phải; phím giữa 2,5,8,0 dùng ngón có khoảng cách Manhattan ngắn hơn. Nếu hòa, dùng hand thuận. Sau mỗi lần bấm, cập nhật vị trí ngón đã dùng. Trả chuỗi L/R.

Input / Output

Nhận numbers và hand là "left" hoặc "right". Trả chuỗi quyết định.

Ràng buộc và lưu ý

- Khoảng cách tính theo số bước lên/xuống/trái/phải. Tie-break chỉ áp dụng khi hai khoảng cách bằng nhau.

Contract

Trả chuỗi L/R theo quy tắc cột và khoảng cách Manhattan.

Pattern và dấu hiệu nhận diện

Coordinate simulation + tie-break theo preference.

State • Transition • Invariant

Map phím→tọa độ; left/right là vị trí ngón hiện tại. Cột ngoài quyết định ngay; cột giữa so distance, hòa dùng hand; sau chọn cập nhật đúng ngón.

Complexity

$O(\text{numbers.length})$, $O(1)$.

Bẫy / edge case

- Tọa độ 0,*,# sai; hòa không theo hand; cập nhật cả hai ngón.

Code JavaScript

```
function solution(numbers, hand) {
    const pos = {
        1: [0, 0],
        2: [0, 1],
        3: [0, 2],
        4: [1, 0],
        5: [1, 1],
        6: [1, 2],
        7: [2, 0],
        8: [2, 1],
        9: [2, 2],
        0: [3, 1],
    },
    dist = (a, b) => Math.abs(a[0] - b[0]) + Math.abs(a[1] - b[1]);
    let L = [3, 0],
        R = [3, 2],
        ans = '';
    for (const x of numbers) {
        let use;
        if (x === 1 || x === 4 || x === 7) use = 'L';
        else if (x === 3 || x === 6 || x === 9) use = 'R';
        else {
            const a = dist(L, pos[x]),
                b = dist(R, pos[x]);
            use = a === b ? (hand === 'left' ? 'L' : 'R') : a < b ? 'L' : 'R';
        }
        ans += use;
        if (use === 'L') L = pos[x];
        else R = pos[x];
    }
    return ans;
}
```

Bài 46 — Game gấp thú (크레인 인형뽑기 게임)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/64061?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

board là máy gấp dạng lưới, 0 là ô trống; mỗi cột chứa búp bê phía trên các ô trống thấp hơn. Mỗi move chọn cột 1-based và gấp búp bê khác 0 ở hàng cao nhất, nếu có. Búp bê được đặt lên đỉnh giỏ. Nếu nó giống búp bê đang ở đỉnh giỏ, cả hai biến mất và cộng 2 vào số lượng đã biến mất. Hãy xử lý mọi move.

Input / Output

Nhận board và moves. Trả tổng số búp bê biến mất.

Ràng buộc và lưu ý

- Sau khi gấp phải đặt ô board thành 0 và dừng quét cột. Một move vào cột rỗng không thay đổi state.

Contract

Đếm số búp bê biến mất khi gấp theo moves.

Pattern và dấu hiệu nhận diện

Matrix column scan + stack cancellation.

State • Transition • Invariant

Mỗi move tìm ô khác 0 đầu cột, set 0. Nếu bằng top stack thì pop và +2, không thì push. Stack là giỏ sau khi xử lý prefix moves.

Complexity

$O(\text{moves} \cdot \text{rows})$, $O(\text{rows} \cdot \text{cols})$ worst-case cho stack.

Bẫy / edge case

- Quên move là 1-based; tiếp tục quét sau khi đã gấp; +1 thay vì +2.

Code JavaScript

```
function solution(board, moves) {
  const st = [];
  let ans = 0;
  for (const m of moves) {
    for (let r = 0; r < board.length; r++) {
      const x = board[r][m - 1];
      if (!x) continue;
      board[r][m - 1] = 0;
      if (st[st.length - 1] === x) {
        st.pop();
        ans += 2;
      } else st.push(x);
      break;
    }
  }
  return ans;
}
```

Bài 47 — Tỷ lệ thất bại (실패율)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/42889?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Có N stage và danh sách stages, trong đó stages[i] là stage người chơi đang dừng lại; N+1 nghĩa đã vượt hết. Tỷ lệ thất bại stage s = số người đang ở s / số người đã tới s. Sắp các stage 1..N theo tỷ lệ giảm dần; nếu hòa, stage nhỏ hơn đứng trước.

Input / Output

Nhận N và stages. Trả mảng số stage theo thứ tự.

Ràng buộc và lưu ý

- Khi không còn người nào tới một stage, tỷ lệ là 0. Cần giảm số người còn lại sau khi tính tỷ lệ stage hiện tại.

Contract

Sắp stage giảm theo failure rate, hòa stage nhỏ trước.

Pattern và dấu hiệu nhận diện

Counting + sort comparator nhiều khóa.

State • Transition • Invariant

Đếm players đứng ở stage. remaining bắt đầu tổng; rate=count[s]/remaining, rồi remaining-=count[s]. Sort theo rate giảm, id tăng.

Complexity

$O(N + \text{players} + N \log N)$, $O(N)$.

Bẫy / edge case

- Chia 0 khi không còn người; cập nhật remaining trước khi tính; tie-break thiếu.

Code JavaScript

```
function solution(N, stages) {
  const c = Array(N + 2).fill(0);
  for (const s of stages) c[s]++;
  let remain = stages.length;
  const a = [];
  for (let s = 1; s <= N; s++) {
    a.push([s, remain ? c[s] / remain : 0]);
    remain -= c[s];
  }
  return a.sort((x, y) => y[1] - x[1] || x[0] - y[0]).map((x) => x[0]);
}
```


Bài 48 — Đồng phục thể dục (체육복)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/42862?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Có n học sinh cần mỗi người một bộ đồng phục. lost là người bị mất, reserve là người có một bộ dư; một người có thể nằm ở cả hai danh sách, khi đó bộ dư bù cho chính họ. Người còn dư chỉ cho học sinh số liền trước hoặc liền sau và chỉ cho một bộ. Hãy tối đa số học sinh có thể học.

Input / Output

Nhận $n, \text{lost}, \text{reserve}$. Trả số học sinh có ít nhất một bộ.

Ràng buộc và lưu ý

- Phải xử lý giao giữa lost và reserve trước hoặc dùng mảng số bộ rỗng. Một chiến lược greedy trái→phải nhất quán là đủ.

Contract

Tối đa số học sinh có đồ sau khi người dư cho hàng xóm.

Pattern và dấu hiệu nhận diện

Greedy trên trạng thái net + ưu tiên nhất quán.

State • Transition • Invariant

Array have khởi tạo 1, lost--, reserve++. Duyệt trái→phải người có 2: cho bên trái thiếu trước, rồi bên phải. State net xử lý đúng người vừa mất vừa dư.

Complexity

$O(n + \text{lost} + \text{reserve})$, $O(n)$.

Bẫy / edge case

- Xử lý Set lost/reserve không loại giao; cho cả hai bên; đổi ưu tiên giữa chừng.

Code JavaScript

```
function solution(n, lost, reserve) {
  const a = Array(n + 2).fill(1);
  a[0] = a[n + 1] = 0;
  for (const x of lost) a[x]--;
  for (const x of reserve) a[x]++;
  for (let i = 1; i <= n; i++)
    if (a[i] === 2) {
      if (a[i - 1] === 0) {
        a[i]--;
        a[i - 1]++;
      } else if (a[i + 1] === 0) {
        a[i]--;
        a[i + 1]++;
      }
    }
  return a.slice(1, n + 1).filter((x) => x > 0).length;
}
```

Bài 49 — Thi thử (모의고사)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/42840?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Ba thí sinh trả lời theo các mẫu lập cố định: [1,2,3,4,5], [2,1,2,3,2,4,2,5], [3,3,1,1,2,2,4,4,5,5]. So với answers, đếm số câu đúng của từng người. Trả mọi số thứ tự 1,2,3 có điểm cao nhất theo thứ tự tăng.

Input / Output

Nhận answers. Trả mảng index thí sinh thắng.

Ràng buộc và lưu ý

- Mẫu lập bằng modulo. Có thể có nhiều người đồng hạng cao nhất.

Contract

Trả index 1-based của mọi người có số câu đúng cao nhất.

Pattern và dấu hiệu nhận diện

Mẫu tuần hoàn + counting + collect ties.

State • Transition • Invariant

Ba pattern; score[i] tăng nếu pattern[i][q%len]==answers[q]. Tìm max rồi lọc score bằng max.

Complexity

$O(3n)$, $O(1)$.

Bẫy / edge case

- Modulo sai chiều; chỉ trả một người; quên index 1-based.

Code JavaScript

```
function solution(answers) {
  const p = [
    [1, 2, 3, 4, 5],
    [2, 1, 2, 3, 2, 4, 2, 5],
    [3, 3, 1, 1, 2, 2, 4, 4, 5, 5],
  ],
  s = [0, 0, 0];
  answers.forEach((x, i) =>
    p.forEach((a, j) => {
      if (a[i % a.length] === x) s[j]++;
    })
  );
  const m = Math.max(...s);
  return s.map((x, i) => (x === m ? i + 1 : 0)).filter(Boolean);
}
```

Bài 50 — Số thứ K (K번째수)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/42748?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi command $[i, j, k]$ dùng chỉ số 1-based: lấy đoạn array từ i tới j inclusive, sắp tăng, rồi chọn phần tử thứ k trong đoạn đã sắp. Thực hiện độc lập cho mọi command.

Input / Output

Nhận array và commands. Trả mảng kết quả.

Ràng buộc và lưu ý

- Không được sort trực tiếp array gốc cho mỗi query; dùng `slice(i-1, j)`. JavaScript sort số cần comparator.

Contract

Với mỗi $[i, j, k]$, lấy đoạn 1-based, sort và trả phần tử k .

Pattern và dấu hiệu nhận diện

Slice + numeric sort + indexing.

State • Transition • Invariant

Chuyển sang `slice(i-1, j)`, sort số tăng, đọc $[k-1]$. Mỗi query độc lập nên không mutate array gốc.

Complexity

$O(q \cdot m \log m)$, $O(m)$.

Bẫy / edge case

- Cận j của slice là exclusive nhưng để inclusive; sort chuỗi; sort array gốc.

Code JavaScript

```
function solution(array, commands) {
  return commands.map(
    ([i, j, k]) => array.slice(i - 1, j).sort((a, b) => a - b)[k - 1],
  );
}
```

Bài 51 — Người chưa hoàn thành (완주하지 못한 선수)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/42576?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

participant chứa tên mọi người dự thi marathon; completion chứa tên mọi người hoàn thành và thiếu đúng một người. Tên có thể trùng, nên mỗi lần xuất hiện đại diện một người khác. Hãy tìm tên người chưa hoàn thành.

Input / Output

Nhận participant, completion. Trả một chuỗi tên.

Ràng buộc và lưu ý

- participant dài hơn completion đúng 1. Set không đủ vì duplicate có ý nghĩa; dùng multiset tần suất hoặc sort.

Contract

Tìm tên participant còn dư sau khi trừ completion, kể cả trùng tên.

Pattern và dấu hiệu nhận diện

Frequency Map / multiset difference.

State • Transition • Invariant

Đếm participant; với mỗi completion giảm. Entry còn count>0 là đáp án. Map biểu diễn multiset, không chỉ membership.

Complexity

$O(n)$, $O(n)$.

Bẫy / edge case

- Dùng Set làm mất duplicate; sort được nhưng $O(n \log n)$; trả key có count 0.

Code JavaScript

```
function solution(participant, completion) {
  const c = new Map();
  for (const x of participant) c.set(x, (c.get(x) ?? 0) + 1);
  for (const x of completion) c.set(x, c.get(x) - 1);
  for (const [x, n] of c) if (n) return x;
}
```

Bài 52 — Game phi tiêu (다트 게임)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/17682?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Chuỗi dartResult mô tả đúng ba lượt. Mỗi lượt gồm điểm 0..10, bonus S/D/T tương ứng lũy thừa 1/2/3, và có thể có option: '*' nhân đôi điểm lượt này và lượt ngay trước nếu có; '#' đổi dấu điểm lượt này. Tính tổng ba lượt sau mọi modifier.

Input / Output

Nhận dartResult. Trả tổng điểm.

Ràng buộc và lưu ý

- Phải parse số 10 như một token. Dấu * tác động ngược lại lượt trước đã lưu; option có thể vắng mặt.

Contract

Tính tổng 3 lượt ném theo điểm, bonus S/D/T và option */#.

Pattern và dấu hiệu nhận diện

Parsing token + stack điểm + modifier tác động phần tử trước.

State • Transition • Invariant

Regex tách mỗi lượt; push score^power rồi xử lý option: * nhân current và previous, # đổi dấu current. Mảng giữ điểm đã hoàn tất từng lượt.

Complexity

$O(|\text{dartResult}|)$, $O(3)$.

Bẫy / edge case

- Đọc 10 thành 1 và 0; * quên lượt trước; áp option trước lũy thừa.

Code JavaScript

```
function solution(s) {
  const a = [];
  for (const m of s.matchAll(/(10|\d)([SDT])([*#]?)/g)) {
    let v = Number(m[1]) ** { S: 1, D: 2, T: 3 }[m[2]];
    if (m[3] === '*') {
      v *= 2;
      if (a.length) a[a.length - 1] *= 2;
    } else if (m[3] === '#') v = -v;
    a.push(v);
  }
  return a.reduce((x, y) => x + y, 0);
}
```

Bài 53 — Bản đồ bí mật (비밀지도)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/17681?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Hai mảng `arr1, arr2` mã hóa từng hàng bản đồ dưới dạng số nhị phân n bit. Một vị trí là tường '#' nếu bit tương ứng bằng 1 ở ít nhất một bản đồ; chỉ khi cả hai bit 0 mới là khoảng trống. Ghép từng hàng thành chuỗi dài n .

Input / Output

Nhận $n, arr1, arr2$. Trả mảng n chuỗi bản đồ.

Ràng buộc và lưu ý

- Dùng OR bit, đổi sang binary và `padStart` đủ n ; giữ khoảng trắng trong output.

Contract

Ghép hai bản đồ nhị phân thành array chuỗi `#/space`.

Pattern và dấu hiệu nhận diện

Bitwise OR + binary padding + char mapping.

State • Transition • Invariant

Mỗi row: `(arr1[i] | arr2[i]).toString(2).padStart(n, '0')`; map $1 \rightarrow \#$, $0 \rightarrow \text{space}$.

Complexity

$O(n^2)$, $O(n^2)$ output.

Bẫy / edge case

- AND thay OR; thiếu `padStart`; regex chỉ thay lần đầu.

Code JavaScript

```
function solution(n, arr1, arr2) {
  return arr1.map((x, i) =>
    (x | arr2[i]).toString(2).padStart(n, '0').replace(/1/g, '#').replace(/0/g, ' '),
  );
}
```

Bài 54 — Ngân sách (예산)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12982?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mỗi phòng ban yêu cầu một khoản ngân sách và chỉ được cấp đủ hoặc không cấp. Với tổng budget, hãy chọn sao cho số phòng ban được cấp là lớn nhất; không cần dùng hết ngân sách.

Input / Output

Nhận d và budget. Trả số phòng ban tối đa.

Ràng buộc và lưu ý

- Chọn yêu cầu nhỏ nhất trước bằng sort tăng. Khi phần tử hiện tại đã vượt ngân sách còn lại, mọi phần tử sau cũng không thể chọn.

Contract

Tối đa số phòng ban được cấp đủ trong budget.

Pattern và dấu hiệu nhận diện

Sort tăng + greedy prefix.

State • Transition • Invariant

Để tối đa count, chọn yêu cầu nhỏ nhất trước. Sort tăng, trừ khi đủ và dừng ở phần tử đầu không đủ; mọi phần tử sau lớn hơn nên cũng không đủ.

Complexity

$O(n \log n)$, $O(1)$ ngoài sort.

Bẫy / edge case

- Sort mặc định; tiếp tục bỏ qua một yêu cầu lớn để tìm nhỏ hơn dù đã sort; cấp một phần.

Code JavaScript

```
function solution(d, budget) {
  d.sort((a, b) => a - b);
  let count = 0;
  for (const x of d) {
    if (x > budget) break;
    budget -= x;
    count++;
  }
  return count;
}
```

Bài 55 — Tạo số nguyên tố (소수 만들기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12977?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Chọn ba phần tử ở ba index khác nhau trong nums. Nếu tổng là số nguyên tố thì tổ hợp đó được tính. Hãy đếm số tổ hợp.

Input / Output

Nhận nums. Trả số tổ hợp có tổng prime.

Ràng buộc và lưu ý

- Độ dài nhỏ, duyệt $i < j < k$. Kiểm tra prime phải loại số nhỏ hơn 2 và thử ước tới căn bậc hai inclusive.

Contract

Đếm bộ ba có tổng là số nguyên tố.

Pattern và dấu hiệu nhận diện

Brute force bộ ba + primality tới sqrt.

State • Transition • Invariant

Duyệt $i < j < k$; isPrime kiểm tra d từ 2 tới sqrt(sum). Tách enumeration và predicate để dễ chứng minh.

Complexity

$O(n^3 \cdot \text{sqrt}(\text{maxSum}))$, $O(1)$.

Bẫy / edge case

- Coi 1 là prime; đếm permutation; kiểm tra $d < \text{sqrt}$ thay vì $\leq$.

Code JavaScript

```
function solution(nums) {
  const prime = (x) => {
    if (x < 2) return false;
    for (let d = 2; d * d <= x; d++) if (x % d === 0) return false;
    return true;
  };
  let ans = 0;
  for (let i = 0; i < nums.length; i++)
    for (let j = i + 1; j < nums.length; j++)
      for (let k = j + 1; k < nums.length; k++)
        if (prime(nums[i] + nums[j] + nums[k])) ans++;
  return ans;
}
```


Bài 56 — In sao hình chữ nhật (직사각형 별찍기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12969?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Đây là bài console I/O: đọc hai số n, m từ stdin và in hình chữ nhật gồm m dòng, mỗi dòng đúng n dấu '*'.

Input / Output

Input một dòng chứa n m . Output m dòng sao, không có separator giữa các sao.

Ràng buộc và lưu ý

- Signature khác các bài solution thông thường. Tránh đảo chiều rộng n và chiều cao m .

Contract

In n dấu * trên m dòng từ input stdin của bài console.

Pattern và dấu hiệu nhận diện

Output construction bằng repeat.

State • Transition • Invariant

Tạo một dòng '*'.repeat(n), lặp m lần và join newline. Đây là bài I/O, signature khác solution thường.

Complexity

$O(nm)$, $O(nm)$ cho chuỗi output.

Bẫy / edge case

- Đảo n, m ; thêm newline thừa thường vẫn được nhưng nên kiểm soát; dùng console.log từng ký tự.

Code JavaScript

```
process.stdin.on('data', (d) => {
  const [n, m] = d.toString().trim().split(/\s+/).map(Number);
  console.log(Array(m).fill('*'.repeat(n)).join('\n'));
});
```

Bài 57 — n số cách nhau x (x만큼 간격이 있는 n개의 숫자)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12954?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tạo dãy n số bắt đầu từ x và mỗi phần tử cách nhau x: x, 2x, ..., nx.

Input / Output

Nhận x, n. Trả mảng dài n.

Ràng buộc và lưu ý

- Phần tử zero-based i là $x \times (i + 1)$; x có thể âm.

Contract

Trả [x, 2x, ..., nx].

Pattern và dấu hiệu nhận diện

Arithmetic sequence + Array.from.

State • Transition • Invariant

Phần tử index i là $x \cdot (i + 1)$; công thức trực tiếp, không cần state phức tạp.

Complexity

O(n), O(n) output.

Bẫy / edge case

- Bắt đầu i=0 cho ra 0; lặp n+1 phần tử.

Code JavaScript

```
function solution(x, n) {  
  return Array.from({ length: n }, (_, i) => x * (i + 1));  
}
```

Bài 58 — Cộng ma trận (행렬의 덧셈)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12950?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Hai ma trận `arr1, arr2` có cùng số hàng và cột. Hãy tạo ma trận mới mà mỗi ô bằng tổng hai ô cùng vị trí.

Input / Output

Nhận `arr1, arr2`. Trả ma trận tổng.

Ràng buộc và lưu ý

- Duyệt đúng `row/column`; không tạo các hàng dùng chung reference nếu tự cấp phát.

Contract

Trả ma trận cùng kích thước với tổng phần tử tương ứng.

Pattern và dấu hiệu nhận diện

Matrix zip theo `row/col`.

State • Transition • Invariant

Map `arr1` theo `i, j` và cộng `arr2[i][j]`. Không dùng `fill(Array)` để tránh `shared row` nếu tự cấp phát.

Complexity

$O(\text{rows} \cdot \text{cols})$, $O(\text{rows} \cdot \text{cols})$ output.

Bẫy / edge case

- Dùng `arr[i, j]`; nhầm số hàng/cột; mutate ngoài ý muốn.

Code JavaScript

```
function solution(arr1, arr2) {  
  return arr1.map((row, i) => row.map((x, j) => x + arr2[i][j]));  
}
```

Bài 59 — Che số điện thoại (핸드폰 번호 가리기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12948?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Ẩn mọi ký tự của số điện thoại trừ bốn ký tự cuối bằng dấu '*'. Giữ nguyên độ dài và thứ tự bốn số cuối.

Input / Output

Nhận phone_number dạng chuỗi. Trả chuỗi đã che.

Ràng buộc và lưu ý

- Độ dài ít nhất 4; không chuyển sang Number vì có thể mất số 0 đầu.

Contract

Đổi mọi ký tự trừ 4 ký tự cuối thành *.

Pattern và dấu hiệu nhận diện

String slicing + repeat.

State • Transition • Invariant

Số sao là length-4; ghép slice(-4). Constraints bảo đảm đủ dài.

Complexity

O(n), O(n).

Bẫy / edge case

- Mask cả 4 số cuối; dùng Number làm mất số 0 đầu.

Code JavaScript

```
function solution(phone_number) {  
    return '*'.repeat(phone_number.length - 4) + phone_number.slice(-4);  
}
```

Bài 60 — Số Harshad (하샤드 수)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12947?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Số x là Harshad nếu chia hết cho tổng các chữ số của chính nó. Hãy kiểm tra x có phải Harshad hay không.

Input / Output

Nhận x . Trả boolean.

Ràng buộc và lưu ý

- Tính tổng từng digit dưới dạng số rồi kiểm tra $x \% \text{sum} === 0$.

Contract

Trả x có chia hết cho tổng chữ số hay không.

Pattern và dấu hiệu nhận diện

Digit reduce + divisibility.

State • Transition • Invariant

Chuyển x thành chữ số, cộng; kiểm tra $x \% \text{sum} === 0$.

Complexity

$O(\log x)$, $O(\log x)$ nếu tạo string.

Bẫy / edge case

- Quên `Number(c)`; chia ngược $\text{sum} \% x$; xử lý 0 dù constraint dương.

Code JavaScript

```
function solution(x) {  
  const s = [...String(x)].reduce((a, c) => a + Number(c), 0);  
  return x % s === 0;  
}
```

Bài 61 — Tính trung bình (평균 구하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12944?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tính trung bình cộng của mọi phần tử trong arr.

Input / Output

Nhận arr số nguyên. Trả số thực hoặc số nguyên tùy kết quả.

Ràng buộc và lưu ý

- Array không rỗng; tổng rồi chia length, không làm tròn.

Contract

Trả trung bình cộng array.

Pattern và dấu hiệu nhận diện

Reduce / aggregate.

State • Transition • Invariant

Tổng bằng reduce với initial 0 rồi chia length.

Complexity

$O(n)$, $O(1)$.

Bẫy / edge case

- Chia trong từng vòng gây sai; reduce thiếu initial không cần thiết; integer division ở ngôn ngữ khác.

Code JavaScript

```
function solution(arr) {  
  return arr.reduce((a, b) => a + b, 0) / arr.length;  
}
```

Bài 62 — Phỏng đoán Collatz (콜라츠 추측)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12943?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Bắt đầu từ num. Nếu chẵn chia 2, nếu lẻ nhân 3 cộng 1; lặp tới khi thành 1. Trả số bước nếu đạt 1 trong không quá 500 phép biến đổi, ngược lại -1. Input 1 trả 0.

Input / Output

Nhận num. Trả số bước hoặc -1.

Ràng buộc và lưu ý

- Dừng sau tối đa 500 transition; không thực hiện vòng đầu khi num đã là 1.

Contract

Trả số phép biến đổi để về 1, hoặc -1 nếu vượt 500.

Pattern và dấu hiệu nhận diện

While simulation với cap số bước.

State • Transition • Invariant

steps=0; trong khi num≠1 và steps<500, áp parity transition rồi tăng. Sau vòng trả steps nếu num==1.

Complexity

O(500), O(1).

Bẫy / edge case

- Input 1 phải trả 0; thực hiện 501 bước; kiểm tra parity sau khi sửa num.

Code JavaScript

```
function solution(num) {
  let n = 0;
  while (num !== 1 && n < 500) {
    num = num % 2 === 0 ? num / 2 : num * 3 + 1;
    n++;
  }
  return num === 1 ? n : -1;
}
```

Bài 63 — Ước chung lớn nhất và bội chung nhỏ nhất (최대공약수와 최소공배수)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12940?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tính ước chung lớn nhất và bội chung nhỏ nhất của hai số tự nhiên n, m . Trả theo đúng thứ tự gcd trước, lcm sau.

Input / Output

Nhận n, m . Trả $[gcd, lcm]$.

Ràng buộc và lưu ý

- Dùng Euclid cho gcd; $lcm = n/gcd \times m$. Cả hai số dương trong ràng buộc đề.

Contract

Trả $[gcd(n, m), lcm(n, m)]$.

Pattern và dấu hiệu nhận diện

Euclid gcd + công thức lcm.

State • Transition • Invariant

Euclid lặp $a, b = [b, a \% b]$ tới $b = 0$. $lcm = n/gcd \cdot m$ để giảm nguy cơ overflow.

Complexity

$O(\log \min(n, m))$, $O(1)$.

Bẫy / edge case

- Nhân trước khi chia; trả sai thứ tự; vòng lặp không cập nhật đồng thời đúng.

Code JavaScript

```
function solution(n, m) {
  let a = n,
      b = m;
  while (b) [a, b] = [b, a % b];
  return [a, (n / a) * m];
}
```


Bài 64 — Chẵn và lẻ (짝수와 홀수)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12937?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Kiểm tra số nguyên num là chẵn hay lẻ và trả đúng chuỗi tiếng Anh theo yêu cầu.

Input / Output

Nhận num. Trả "Even" nếu chẵn, "Odd" nếu lẻ.

Ràng buộc và lưu ý

- Số có thể âm hoặc 0; kiểm tra $\text{num} \% 2 === 0$ tránh vấn đề số âm lẻ có remainder -1 trong JavaScript.

Contract

Trả Even nếu num chẵn, Odd nếu lẻ.

Pattern và dấu hiệu nhận diện

Parity bằng modulo.

State • Transition • Invariant

$\text{num} \% 2 === 0$ là điều kiện trực tiếp, hoạt động cả số âm trong JS cho test bằng 0.

Complexity

$O(1)$, $O(1)$.

Bẫy / edge case

- Dùng $\text{num} \% 2 === 1$ làm sai số âm lẻ; đảo chuỗi output.

Code JavaScript

```
function solution(num) {  
  return num % 2 === 0 ? 'Even' : 'Odd';  
}
```

Bài 65 — Loại số nhỏ nhất (제일 작은 수 제거하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12935?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Loại phần tử nhỏ nhất khỏi arr và giữ nguyên thứ tự các phần tử còn lại. Nếu arr chỉ có một phần tử thì sau khi loại phải trả [-1].

Input / Output

Nhận arr. Trả mảng kết quả.

Ràng buộc và lưu ý

- Theo đề các giá trị khác nhau, nên filter theo min chỉ loại một phần tử. Không sort vì phải giữ thứ tự.

Contract

Bỏ phần tử nhỏ nhất; nếu kết quả rỗng trả [-1].

Pattern và dấu hiệu nhận diện

Scan min + filter một giá trị.

State • Transition • Invariant

Tìm min bằng Math.min; filter $x \neq \text{min}$. Để bảo các số khác nhau nên loại đúng một phần tử.

Complexity

$O(n)$, $O(n)$ output.

Bẫy / edge case

- Mutate bằng splice sai index; quên [-1]; với bài biến thể duplicate cần đọc lại contract.

Code JavaScript

```
function solution(arr) {  
  if (arr.length === 1) return [-1];  
  const m = Math.min(...arr);  
  return arr.filter(x => x !== m);  
}
```

Bài 66 — Kiểm tra căn bậc hai nguyên (정수 제곱근 판별)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12934?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Nếu n là bình phương hoàn hảo của số tự nhiên x , trả bình phương của $x+1$; nếu không, trả -1 .

Input / Output

Nhận n . Trả $(\text{sqrt}(n)+1)^2$ hoặc -1 .

Ràng buộc và lưu ý

- Kiểm tra căn bậc hai là số nguyên; không chỉ floor rồi mặc định hợp lệ.

Contract

Nếu n là bình phương của x thì trả $(x+1)^2$, nếu không trả -1 .

Pattern và dấu hiệu nhận diện

Integer square root test.

State • Transition • Invariant

$\text{root} = \text{Math.sqrt}(n)$; $\text{Number.isInteger}(\text{root})$ quyết định. Bound đủ an toàn với Number .

Complexity

$O(1)$, $O(1)$.

Bẫy / edge case

- Dùng floor rồi luôn coi là square; trả $\text{root}+1$ thay vì bình phương.

Code JavaScript

```
function solution(n) {  
  const r = Math.sqrt(n);  
  return Number.isInteger(r) ? (r + 1) ** 2 : -1;  
}
```

Bài 67 — Sắp chữ số giảm dần (정수 내림차순으로 배치하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12933?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tách các chữ số của số tự nhiên n , sắp theo thứ tự giảm dần rồi ghép lại thành một số.

Input / Output

Nhận n . Trả số sau sắp.

Ràng buộc và lưu ý

- Sort ký tự số bằng comparator số hoặc thứ tự digit; output theo đề là Number.

Contract

Sắp chữ số của n giảm dần và trả số.

Pattern và dấu hiệu nhận diện

Representation string + sort comparator.

State • Transition • Invariant

String $\rightarrow$ array digits; $\text{sort}((a,b) \Rightarrow b-a)$; join và Number.

Complexity

$O(d \log d)$, $O(d)$.

Bẫy / edge case

- Sort tăng; trả string sai kiểu; Number an toàn theo bound.

Code JavaScript

```
function solution(n) {  
    return Number([...String(n)].sort((a, b) => b - a).join(''));  
}
```

Bài 68 — Đảo số tự nhiên thành mảng (자연수 뒤집어 배열로 만들기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12932?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tách các chữ số của n theo thứ tự từ hàng đơn vị tới hàng cao nhất. Ví dụ 12345 thành [5,4,3,2,1].

Input / Output

Nhận n. Trả mảng digit.

Ràng buộc và lưu ý

- Mỗi phần tử output là số, không phải chuỗi.

Contract

Trả các chữ số của n theo thứ tự ngược.

Pattern và dấu hiệu nhận diện

String reverse + digit mapping.

State • Transition • Invariant

String→spread→reverse→map Number.

Complexity

$O(d)$, $O(d)$.

Bẫy / edge case

- Trả string digits; sort thay reverse; dùng arithmetic nhưng đẩy sai thứ tự.

Code JavaScript

```
function solution(n) {  
  return [...String(n)].reverse().map(Number);  
}
```

Bài 69 — Tổng chữ số (자릿수 더하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12931?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tính tổng mọi chữ số của số tự nhiên N.

Input / Output

Nhận N. Trả tổng chữ số.

Ràng buộc và lưu ý

- Khi dùng String phải chuyển từng ký tự thành Number để tránh nối chuỗi.

Contract

Tính tổng chữ số của số tự nhiên N.

Pattern và dấu hiệu nhận diện

Digit reduce.

State • Transition • Invariant

Chuyển String và reduce Number(c); accumulator là tổng prefix chữ số.

Complexity

O(d), O(d) do string.

Bẫy / edge case

- Nối chuỗi do quên Number; dùng parseInt cả chuỗi.

Code JavaScript

```
function solution(n) {  
  return [...String(n)].reduce((s, c) => s + Number(c), 0);  
}
```

Bài 70 — Tạo chuỗi kỳ lạ (이상한 문자 만들기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12930?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Với mỗi từ trong s, ký tự ở vị trí chẵn bên trong từ đổi thành chữ hoa và vị trí lẻ đổi thành chữ thường. Khoảng trắng ngăn từ và phải được giữ nguyên; sau mỗi khoảng trắng, index trong từ reset về 0. Đầu vào có thể có nhiều khoảng trắng liên tiếp.

Input / Output

Nhận s. Trả chuỗi biến đổi.

Ràng buộc và lưu ý

- Index tính riêng cho từng từ, không dùng index toàn chuỗi. Cần bảo toàn chính xác số lượng/vị trí khoảng trắng.

Contract

Trong mỗi từ, index chẵn viết hoa, lẻ viết thường; khoảng trắng reset index.

Pattern và dấu hiệu nhận diện

Parsing theo word position, giữ nguyên spaces.

State • Transition • Invariant

Duyệt từng char với local index k; gặp space thì reset k=0, nếu không transform theo k rồi k++. Cách này giữ cả nhiều spaces.

Complexity

O(n), O(n).

Bẫy / edge case

- Dùng split(' ') rồi làm mất/khó giữ spaces; dùng global index thay index trong word.

Code JavaScript

```
function solution(s) {
  let k = 0,
      out = '';
  for (const c of s) {
    if (c === ' ') {
      out += c;
      k = 0;
    } else {
      out += k % 2 === 0 ? c.toUpperCase() : c.toLowerCase();
      k++;
    }
  }
  return out;
}
```

Bài 71 — Tổng các ước (약수의 합)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12928?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tính tổng tất cả ước dương của n .

Input / Output

Nhận n . Trả tổng ước.

Ràng buộc và lưu ý

- Duyệt d tới căn n và cộng theo cặp $d, n/d$; nếu $d^2 = n$ chỉ cộng d một lần.

Contract

Trả tổng mọi ước của n .

Pattern và dấu hiệu nhận diện

Divisor pairing tới $\sqrt{n}$.

State • Transition • Invariant

Với $d|n$, cộng d và n/d ; nếu $d^2 = n$ chỉ cộng một lần.

Complexity

$O(\sqrt{n})$, $O(1)$.

Bẫy / edge case

- Đếm square root hai lần; vòng $d < n$ bỏ n ; khởi tạo sai cho $n=1$.

Code JavaScript

```
function solution(n) {  
  let sum = 0;  
  for (let d = 1; d * d <= n; d++) if (n % d === 0) sum += d * d === n ? d : d + n / d;  
  return sum;  
}
```


Bài 72 — Mã Caesar (시저 암호)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12926?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mã Caesar dịch mỗi chữ cái trong s tiến n vị trí trong alphabet, quay vòng sau z/Z. Bảo toàn chữ hoa/chữ thường và giữ nguyên khoảng trắng.

Input / Output

Nhận s,n. Trả chuỗi đã dịch.

Ràng buộc và lưu ý

- Chỉ chữ Latin và space theo đề. Modulo 26 phải tính tương đối với base A hoặc a.

Contract

Dịch mỗi chữ cái n bước, space giữ nguyên.

Pattern và dấu hiệu nhận diện

Character code + modulo, bảo toàn space và case.

State • Transition • Invariant

Xác định base 65/97 theo case; $code' = base + (code - base + n) \% 26$. Map từng ký tự.

Complexity

$O(|s|)$, $O(|s|)$.

Bẫy / edge case

- Dịch space; modulo sai base; chỉ xử lý lowercase.

Code JavaScript

```
function solution(s, n) {
  return [...s]
    .map((c) => {
      if (c === ' ') return c;
      const b = c <= 'Z' ? 65 : 97;
      return String.fromCharCode(b + ((c.charCodeAt(0) - b + n) % 26));
    })
    .join('');
}
```

Bài 73 — Đổi chuỗi thành số nguyên (문자열을 정수로 바꾸기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12925?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Chuyển chuỗi s biểu diễn một số nguyên, có thể bắt đầu bằng '+' hoặc '-', thành giá trị số nguyên tương ứng.

Input / Output

Nhận s. Trả Number.

Ràng buộc và lưu ý

- Chuỗi luôn hợp lệ và nằm trong bound nhỏ; dấu là một phần của representation.

Contract

Chuyển chuỗi có thể có dấu +/- thành integer.

Pattern và dấu hiệu nhận diện

Built-in numeric parsing.

State • Transition • Invariant

Number(s) diễn tả trực tiếp contract và xử lý dấu.

Complexity

$O(|s|)$, $O(1)$.

Bẫy / edge case

- Dùng parseInt rồi quên rằng contract đảm bảo format; tự xử lý dấu để off-by-one.

Code JavaScript

```
function solution(s) {  
    return Number(s);  
}
```

Bài 74 — Chuỗi dưa hấu (수박수박수박수박수박수?)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12922?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tạo chuỗi dài n bằng cách lặp hai ký tự Hàn “수박”, bắt đầu bằng “수”. Ví dụ n=3 cho “수박수”.

Input / Output

Nhận n. Trả chuỗi dài n.

Ràng buộc và lưu ý

- Dùng repeat đủ dài rồi slice đúng n ký tự.

Contract

Trả n ký tự luân phiên 수,박 bắt đầu bằng 수.

Pattern và dấu hiệu nhận diện

Periodic pattern.

State • Transition • Invariant

Lặp '수박' ceil(n/2) lần rồi slice n, hoặc map parity.

Complexity

O(n), O(n).

Bẫy / edge case

- Bắt đầu sai ký tự; slice theo byte thay vì ký tự ở ngôn ngữ khác.

Code JavaScript

```
function solution(n) {  
  return '\uC218\uBC15'.repeat(Math.ceil(n / 2)).slice(0, n);  
}
```

Bài 75 — Đếm số nguyên tố (소수 찾기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12921?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Đếm số nguyên tố trong đoạn từ 1 tới n inclusive.

Input / Output

Nhận n. Trả số lượng prime $\leq n$.

Ràng buộc và lưu ý

- N có thể tới 1.000.000 nên cần sieve. 0 và 1 không phải prime; khi sàng p, bắt đầu đánh dấu từ p^2 .

Contract

Đếm prime không lớn hơn n.

Pattern và dấu hiệu nhận diện

Sieve of Eratosthenes.

State • Transition • Invariant

$\text{prime}[0]=\text{prime}[1]=\text{false}$; với $p^2 \leq n$ nếu $\text{prime}[p]$, đánh dấu từ p^2 bước p. Invariant: trước p, mọi composite có prime factor nhỏ hơn p đã bị loại.

Complexity

$O(n \log \log n)$, $O(n)$.

Bẫy / edge case

- Bắt đầu đánh dấu từ $2p$ vẫn đúng nhưng thừa; coi 1 là prime; dùng nested primality $O(n\sqrt{n})$ có thể chậm.

Code JavaScript

```
function solution(n) {
  const a = Array(n + 1).fill(true);
  a[0] = a[1] = false;
  for (let p = 2; p * p <= n; p++)
    if (a[p]) for (let x = p * p; x <= n; x += p) a[x] = false;
  return a.filter(Boolean).length;
}
```

Bài 76 — Tìm anh Kim ở Seoul (서울에서 김서방 찾기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12919?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Mảng seoul chứa đúng một chuỗi "Kim". Tìm index zero-based của nó và chèn vào câu kết quả đúng định dạng tiếng Hàn: "김서방은 x에 있다".

Input / Output

Nhận seoul. Trả chuỗi theo mẫu.

Ràng buộc và lưu ý

- Đúng một Kim được đảm bảo. Phải giữ nguyên spacing và chữ trong output.

Contract

Trả câu đúng format với index của 'Kim'.

Pattern và dấu hiệu nhận diện

Linear search bằng indexOf.

State • Transition • Invariant

indexOf phù hợp vì contract bảo đảm đúng một Kim; template literal giữ format.

Complexity

$O(n)$, $O(1)$.

Bẫy / edge case

- Trả index thay vì câu; sai spacing/case; find trả value không phải index.

Code JavaScript

```
function solution(seoul) {  
  return ` ${seoul.indexOf('Kim')} `;  
}
```

Bài 77 — Xử lý chuỗi cơ bản (문자열 다루기 기본)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12918?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Kiểm tra s có độ dài đúng 4 hoặc 6 và mọi ký tự đều là chữ số 0-9.

Input / Output

Nhận s. Trả boolean.

Ràng buộc và lưu ý

- Regex phải neo cả chuỗi; không dùng Number đơn thuần vì ký hiệu số như dấu hay exponent không thỏa yêu cầu ký tự.

Contract

Trả true khi s dài 4 hoặc 6 và chỉ gồm chữ số.

Pattern và dấu hiệu nhận diện

Validation length + regex toàn chuỗi.

State • Transition • Invariant

Kiểm tra length trước; regex `/^\d+$/` neo cả hai đầu. Không dùng Number vì exponent/dấu có thể được chấp nhận ngoài ý muốn.

Complexity

$O(|s|)$, $O(1)$.

Bẫy / edge case

- Regex thiếu anchors; `Number('1e22')` hợp lệ số nhưng không phải toàn digit; quên một độ dài.

Code JavaScript

```
function solution(s) {  
  return (s.length === 4 || s.length === 6) && /^\d+$/.test(s);  
}
```

Bài 78 — Sắp chuỗi giảm dần (문자열 내림차순으로 배치하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12917?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Sắp các ký tự của chuỗi s theo thứ tự giảm dần theo thứ tự ký tự mà đề sử dụng; trong dữ liệu Latin của bài, chữ thường đứng trước chữ hoa khi sort giảm.

Input / Output

Nhận s. Trả chuỗi đã sắp.

Ràng buộc và lưu ý

- Không thay đổi ký tự, chỉ đổi thứ tự. Default code-point sort rồi reverse phù hợp input chính thức.

Contract

Trả ký tự của s theo thứ tự giảm dần; lowercase đứng trước uppercase theo thứ tự yêu cầu.

Pattern và dấu hiệu nhận diện

Character sort theo code point.

State • Transition • Invariant

Spread, sort(), reverse(), join. Default lexicographic phù hợp bảng ký tự của input Latin.

Complexity

$O(n \log n)$, $O(n)$.

Bẫy / edge case

- Comparator locale thay đổi thứ tự; quên reverse; sort trực tiếp string không được.

Code JavaScript

```
function solution(s) {  
  return [...s].sort().reverse().join('');  
}
```

Bài 79 — Đếm p và y (문자열 내 p와 y의 개수)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12916?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Đếm chữ p và y trong s không phân biệt hoa thường. Trả true nếu hai số lượng bằng nhau; nếu cả hai đều không xuất hiện cũng trả true.

Input / Output

Nhận s. Trả boolean.

Ràng buộc và lưu ý

- Nên lowercase trước hoặc xử lý cả hai case; không coi trường hợp 0-0 là false.

Contract

Không phân biệt hoa thường, trả số p bằng số y.

Pattern và dấu hiệu nhận diện

Case normalization + counting.

State • Transition • Invariant

Lowercase một lần; reduce delta +1 cho p, -1 cho y; kết quả 0 nghĩa bằng nhau.

Complexity

O(n), O(1).

Bẫy / edge case

- So case-sensitive; yêu cầu khi cả hai 0 vẫn true; regex match null.

Code JavaScript

```
function solution(s) {
  let d = 0;
  for (const c of s.toLowerCase()) {
    if (c === 'p') d++;
    else if (c === 'y') d--;
  }
  return d === 0;
}
```


Bài 80 — Sắp chuỗi theo ý muốn (문자열 내 마음대로 정렬하기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12915?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Sắp mảng strings tăng theo ký tự tại index n. Nếu hai chuỗi có cùng ký tự ở n, so toàn bộ chuỗi theo thứ tự từ điển tăng. Trả mảng đã sắp.

Input / Output

Nhận strings, n. Trả mảng chuỗi.

Ràng buộc và lưu ý

- Comparator có hai khóa và phải trả số âm/0/dương, không trả boolean. Mọi chuỗi đủ dài để có index n.

Contract

Sort strings theo ký tự index n; hòa thì lexicographic toàn chuỗi.

Pattern và dấu hiệu nhận diện

Comparator hai khóa.

State • Transition • Invariant

Comparator: a[n] khác thì locale/code comparison; bằng thì compare a,b. Dùng điều kiện rõ để tránh comparator boolean.

Complexity

$O(m \log m \cdot L)$, $O(m)$ tùy engine.

Bẫy / edge case

- Chỉ sort theo a[n] làm tie không xác định; comparator trả true/false; mutate khi còn cần input.

Code JavaScript

```
function solution(strings, n) {
    return strings.sort((a, b) =>
        a[n] === b[n] ? (a < b ? -1 : a > b ? 1 : 0) : a[n] < b[n] ? -1 : 1,
    );
}
```

Bài 81 — Tổng giữa hai số nguyên (두 정수 사이의 합)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12912?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Tính tổng tất cả số nguyên từ a tới b inclusive; a có thể lớn hơn b.

Input / Output

Nhận a,b. Trả tổng.

Ràng buộc và lưu ý

- Dùng $lo = \min(a, b)$, $hi = \max(a, b)$ và công thức cấp số cộng với số phần tử $hi - lo + 1$.

Contract

Tính tổng mọi integer từ a tới b inclusive, không biết thứ tự.

Pattern và dấu hiệu nhận diện

Arithmetic series với min/max.

State • Transition • Invariant

$lo = \min$, $hi = \max$; $count = hi - lo + 1$; $sum = (lo + hi) \cdot count / 2$.

Complexity

$O(1)$, $O(1)$.

Bẫy / edge case

- Quên inclusive +1; giả sử $a \leq b$; $lo + hi$ có thể âm nhưng công thức vẫn đúng.

Code JavaScript

```
function solution(a, b) {  
  const lo = Math.min(a, b),  
        hi = Math.max(a, b);  
  return ((lo + hi) * (hi - lo + 1)) / 2;  
}
```

Bài 82 — Mảng số chia hết (나누어 떨어지는 숫자 배열)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12910?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Lọc các phần tử của arr chia hết cho divisor, sắp tăng dần và trả mảng. Nếu không có phần tử nào, trả [-1].

Input / Output

Nhận arr, divisor. Trả mảng số.

Ràng buộc và lưu ý

- Điều kiện chia hết là $x \% divisor === 0$; numeric sort bắt buộc trong JavaScript.

Contract

Trả các số chia hết divisor tăng dần; nếu không có trả [-1].

Pattern và dấu hiệu nhận diện

Filter + numeric sort + fallback.

State • Transition • Invariant

Filter modulo 0; sort numeric; kiểm tra length để fallback.

Complexity

$O(n \log n)$, $O(n)$.

Bẫy / edge case

- Sort mặc định; trả [] thay [-1]; điều kiện modulo truthy bị đảo.

Code JavaScript

```
function solution(arr, divisor) {  
  const a = arr.filter((x) => x % divisor === 0).sort((x, y) => x - y);  
  return a.length ? a : [-1];  
}
```

Bài 83 — Không thích số giống nhau (같은 숫자는 싫어)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12906?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Loại các số trùng liên tiếp trong arr sao cho mỗi run chỉ còn một phần tử; các giá trị giống nhau nhưng bị ngăn bởi giá trị khác vẫn phải giữ. Ví dụ [1,1,3,3,0,1,1] thành [1,3,0,1].

Input / Output

Nhận arr. Trả mảng đã nén run.

Ràng buộc và lưu ý

- Không dùng Set vì Set xóa duplicate không liên tiếp. So với phần tử cuối output hoặc phần tử ngay trước input.

Contract

Xoá các phần tử liên tiếp trùng nhau, giữ một đại diện.

Pattern và dấu hiệu nhận diện

Run-length compression / stack top.

State • Transition • Invariant

Duyệt arr; push x khi output cuối khác x. Invariant: output là bản nén đúng của prefix.

Complexity

O(n), O(n) output.

Bẫy / edge case

- Dùng Set xoá cả duplicate không liên tiếp; so với arr[i-1] nhưng xử lý index 0 vụng.

Code JavaScript

```
function solution(arr) {
  const out = [];
  for (const x of arr) if (out[out.length - 1] !== x) out.push(x);
  return out;
}
```

Bài 84 — Lấy ký tự giữa (가운데 글자 가져오기)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12903?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Nếu s dài lẻ, trả ký tự chính giữa; nếu dài chẵn, trả hai ký tự giữa.

Input / Output

Nhận s. Trả chuỗi dài 1 hoặc 2.

Ràng buộc và lưu ý

- Index zero-based phải xử lý cả hai parity; chuỗi không rỗng.

Contract

Trả 1 ký tự giữa nếu lẻ, 2 ký tự giữa nếu chẵn.

Pattern và dấu hiệu nhận diện

Index arithmetic + slice.

State • Transition • Invariant

`start=floor((n-1)/2); length=2-n%2. slice(start,start+length).`

Complexity

O(1) theo index, O(1) output.

Bẫy / edge case

- Chẵn lấy hai ký tự lệch phải; dùng substring cận nhầm.

Code JavaScript

```
function solution(s) {  
  const i = Math.floor((s.length - 1) / 2);  
  return s.slice(i, i + (s.length % 2 ? 1 : 2));  
}
```

Bài 85 — Năm 2016 (2016년)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/12901?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Năm 2016 là năm nhuận và ngày 01/01/2016 là thứ Sáu. Với tháng a và ngày b hợp lệ, hãy trả tên thứ dạng SUN, MON, TUE, WED, THU, FRI, SAT.

Input / Output

Nhận a, b. Trả chuỗi ba chữ cái.

Ràng buộc và lưu ý

- Tháng 2 có 29 ngày; tổng số ngày trước ngày cần hỏi dùng b-1 để 01/01 có offset 0.

Contract

Trả thứ trong tuần của ngày a/b năm nhuận 2016.

Pattern và dấu hiệu nhận diện

Modulo ngày trong tuần + prefix month days.

State • Transition • Invariant

Tổng ngày trước tháng a cộng b-1; $\text{index} = (\text{offset} + 5) \% 7$ vì 01/01 là Friday. Array weekday bắt đầu SUN.

Complexity

$O(12)$, $O(1)$.

Bẫy / edge case

- Quên leap February 29; dùng b thay b-1; lệch base weekday.

Code JavaScript

```
function solution(a, b) {
  const md = [31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31],
    w = ['SUN', 'MON', 'TUE', 'WED', 'THU', 'FRI', 'SAT'];
  let d = b - 1;
  for (let m = 1; m < a; m++) d += md[m - 1];
  return w[(5 + d) % 7];
}
```

Bài 86 — Ponketmon (폰켓몬)

Nguồn chính thức: <https://school.programmers.co.kr/learn/courses/30/lessons/1845?language=javascript>

Đề bài — diễn giải tiếng Việt đầy đủ ý

Có `nums.length` Pokémon nhưng chỉ được chọn đúng `nums.length/2` con. Mục tiêu là tối đa số loài khác nhau trong số đã chọn. Hãy trả số loài tối đa có thể đạt.

Input / Output

Nhận `nums`, mỗi số là mã loài. Trả một số nguyên.

Ràng buộc và lưu ý

- Đáp án là $\min(\text{số loại duy nhất}, \text{số slot được chọn})$. `nums` có độ dài chẵn theo đề.

Contract

Chọn tối đa $n/2$ con với nhiều loại khác nhau nhất.

Pattern và dấu hiệu nhận diện

Set cardinality + capacity cap.

State • Transition • Invariant

Số loại khả dụng là Set size, số slot là `nums.length/2`; đáp án $\min$ hai giá trị.

Complexity

$O(n)$, $O(n)$.

Bẫy / edge case

- Đếm tần suất không cần; trả Set size dù vượt slot; dùng ceil thay floor dù n chẵn theo đề.

Code JavaScript

```
function solution(nums) {  
    return Math.min(new Set(nums).size, nums.length / 2);  
}
```

MA TRẬN COVERAGE 84 BÀI

2 bài nằm ở Phần I + 82 thẻ ở Phần II = 84 bài Level 1 hỗ trợ JavaScript theo roster ngày 03/08/2026.

Đã phân tích sâu ở Phần I

250137 — Bảng bố; 340213 — Trình phát video.

Đã chuẩn hoá ở Phần II

- 468371 — Đền giao thông vàng (노란불 신호등); 468370 — Chống spoil từ quan trọng (중요한 단어를 스포 방지); 389478 — Lấy hộp hàng (택배 상자 꺼내기); 388351 — Chế độ làm việc linh hoạt (유연근무제); 258712 — Người nhận quà nhiều nhất (가장 많이 받은 선물); 178871 — Cuộc đua chạy (달리기 경주); 176963 — Điểm kỷ niệm (추억 점수); 172928 — Đi dạo công viên (공원 산책).
- 161990 — Dọn hình nền (바탕화면 정리); 161989 — Sơn lại (덧칠하기); 160586 — Bàn phím được tạo đại khái (대충 만든 자판); 159994 — Chống thẻ (카드 뭉치); 155652 — Mật mã của hai người (둘만의 암호); 150370 — Thời hạn lưu thông tin cá nhân (개인정보 수집 유효기간); 147355 — Chuỗi con có giá trị nhỏ (크기가 작은 부분 문자열); 142086 — Chữ giống gần nhất (가장 가까운 같은 글자).
- 140108 — Tách chuỗi (문자열 나누기); 138477 — Hall of Fame (1) (명예의 전당 (1)); 136798 — Vũ khí hiệp sĩ (기사단원의 무기); 135808 — Người bán trái cây (과일 장수); 134240 — Cuộc thi Food Fight (푸드 파이트 대회); 133502 — Làm hamburger (햄버거 만들기); 133499 — Bập bẹ (2) (옹알이 (2)); 132267 — Bài toán cola (콜라 문제).
- 131705 — Bộ ba số 0 (삼총사); 131128 — Cặp số (숫자 짝꿍); 118666 — Trắc nghiệm tính cách (성격 유형 검사하기); 92334 — Nhận kết quả báo cáo (신고 결과 받기); 87389 — Tìm số có dư 1 (나머지가 1이 되는 수 찾기); 86491 — Hình chữ nhật nhỏ nhất (최소직사각형); 86051 — Cộng số còn thiếu (없는 숫자 더하기); 82612 — Tính số tiền thiếu (부족한 금액 계산하기).
- 81301 — Chuỗi số và từ tiếng Anh (숫자 문자열과 영단어); 77884 — Số lượng ước và phép cộng (약수의 개수와 덧셈); 77484 — Hạng cao nhất và thấp nhất Lotto (로또의 최고 순위와 최저 순위); 76501 — Cộng âm dương (음양 더하기); 72410 — Gợi ý ID mới (신규 아이디 추천); 70128 — Tích vô hướng (내적); 68935 — Đảo số hệ tam phân (3진법 뒤집기); 68644 — Chọn hai số rồi cộng (두 개 뽑아서 더하기).
- 67256 — Bấm bàn phím số (키패드 누르기); 64061 — Game gấp thú (크레인 인형뽑기 게임); 42889 — Tỷ lệ thất bại (실패율); 42862 — Đồng phục thể dục (체육복); 42840 — Thi thử (모의고사); 42748 — Số thứ K (K번째수); 42576 — Người chưa hoàn thành (완주하지 못한 선수); 17682 — Game phi tiêu (다트 게임).
- 17681 — Bản đồ bí mật (비밀지도); 12982 — Ngân sách (예산); 12977 — Tạo số nguyên tố (소수 만들기); 12969 — In sao hình chữ nhật (직사각형 별찍기); 12954 — n số cách nhau x (x만큼 간격이 있는 n개의 숫자); 12950 — Cộng ma trận (행렬의 덧셈); 12948 — Che số điện thoại (핸드폰 번호 가리기); 12947 — Số Harshad (하샤드 수).
- 12944 — Tính trung bình (평균 구하기); 12943 — Phỏng đoán Collatz (콜라츠 추측); 12940 — Ước chung lớn nhất và bội chung nhỏ nhất (최대공약수와 최소공배수); 12937 — Chẵn và lẻ (짝수와 홀수); 12935 — Loại số nhỏ nhất (제일 작은 수 제거하기); 12934 — Kiểm tra căn bậc hai nguyên (정수 제곱근 판별); 12933 — Sắp chữ số giảm dần (정수 내림차순으로 배치하기); 12932 — Đảo số tự nhiên thành mảng (자연수 뒤집어 배열로 만들기).
- 12931 — Tổng chữ số (자릿수 더하기); 12930 — Tạo chuỗi kỳ lạ (이상한 문자 만들기); 12928 — Tổng các ước (약수의 합); 12926 — Mã Caesar (시저 암호); 12925 — Đổi chuỗi thành số nguyên (문자열을 정수로 바꾸기); 12922 — Chuỗi dưa hấu (수박수박수박수박수박수?); 12921 — Đếm số nguyên tố (소수 찾기); 12919 — Tìm anh Kim ở Seoul (서울에서 김서방 찾기).
- 12918 — Xử lý chuỗi cơ bản (문자열 다루기 기본); 12917 — Sắp chuỗi giảm dần (문자열 내림차순으로 배치하기); 12916 — Đếm p và y (문자열 내 p와 y의 개수); 12915 — Sắp chuỗi theo ý muốn (문자열 내 마음대로 정렬하기); 12912 — Tổng giữa hai số nguyên (두 정수 사이의 합); 12910 — Mảng số chia hết (나누어 떨어지는 숫자 배열); 12906 — Không thích số giống nhau (같은 숫자는 싫어); 12903 — Lấy ký tự giữa (가운데 글자 가져오기).
- 12901 — Năm 2016 (2016년); 1845 — Ponketmon (폰켓몬).

Checklist pattern trước khi thi

- Đề hỏi “đã thấy/chưa?” → Set; hỏi “bao nhiêu?” → Map tần suất.
- Nhiều truy vấn lặp lại → tiền xử lý Map/array một lần.
- Lệnh theo thứ tự → state machine; viết transition trước code.
- Khoảng thời gian/giờ → đổi sang phút hoặc giây trước khi tính.
- Sort có tie-break → comparator nhiều khóa, nhớ sort() làm mutate.
- Xoá mẫu ở cuối prefix → stack.
- n nhỏ và chọn 2/3 phần tử → brute force hữu hạn thường là đáp án.
- Số nguyên lớn trong tổng/công thức → kiểm tra Number.MAX_SAFE_INTEGER.
- Grid → thống nhất row/col và kiểm tra biên trước khi truy cập.
- Không nói được invariant → chưa nên gõ code.

Nguồn và phạm vi

Danh mục: <https://school.programmers.co.kr/learn/challenges?levels=1&languages=javascript>. Mọi mô tả trong tài liệu là diễn giải tiếng Việt phục vụ học tập, không sao chép nguyên văn dài từ đề.